计算机网络

西南交通大学 Southwest Jiaotong University

第六章 传输层

授课教师：张新有

Email：Ben_eric@qq.com

计算机与人工智能学院 School of Computing and Artificial Intelligence

本章目标

西南交通大学 Southwest Jiaotong University

- 掌握传输层服务原理
  - 传输层复用和分用（port）
  - 可靠数据传输
  - 流量控制
  - 拥塞控制
- 掌握因特网传输层协议：
  - UDP协议
  - TCP协议
- 掌握套接字编程

本章内容

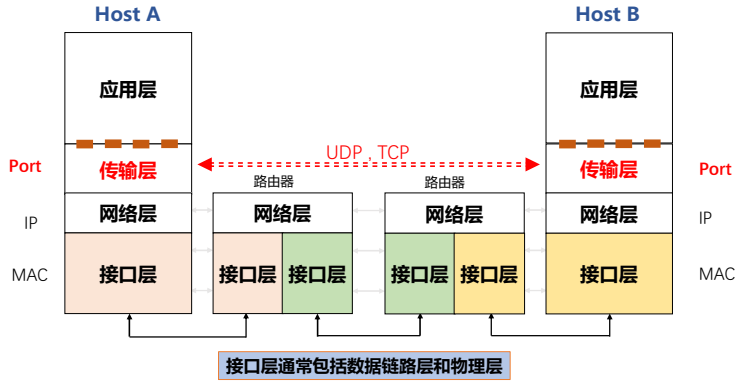
西南交通大学 Southwest Jiaotong University

6.1 传输层服务(p3)
6.2 套接字编程(p10)
6.3 传输层复用和分用(p37)
6.4 无连接传输：UDP(p47)
6.5 面向连接的传输：TCP(p55)
6.6 TCP拥塞控制(p100)
6.7 拥塞控制的发展
6.8 传输层协议的发展



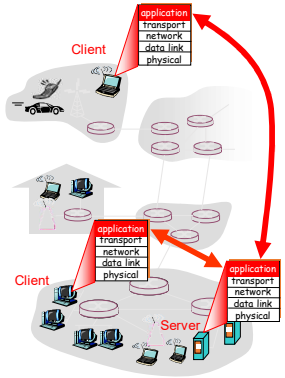
1. 传输层服务

西南交通大学 Southwest Jiaotong University



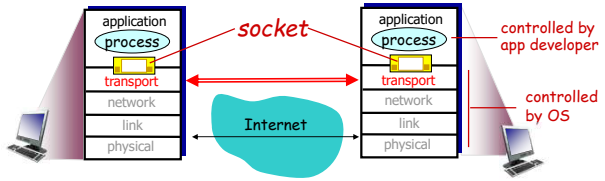
传输层的位置

- 传输层位于应用层和网络层之间：
  - 基于网络层提供的服务，向分布式应用程序（或应用层协议）提供通信服务
- 按照因特网的“端到端”设计原则：
  - 应用程序只运行在终端上，即不需要为网络设备编写应用程序
- 站在应用程序的角度：
  - 传输层应提供网络进程之间本地通信的抽象：即运行在不同终端上的应用进程仿佛是直接连在一起



2. 不同终端上的进程如何通信?

- 设想在应用程序和网络之间存在一扇“门”：
  - 需要发送报文时：发送进程将报文推到门外
  - 门外的运输设施（Internet）将报文送到接收进程的门口
  - 需要接收报文时：接收进程打开门，即可收到报文
- 在TCP/IP网络中，这扇“门”称为套接字（socket），是应用层和传输层的接口，也是应用程序和网络之间的API。

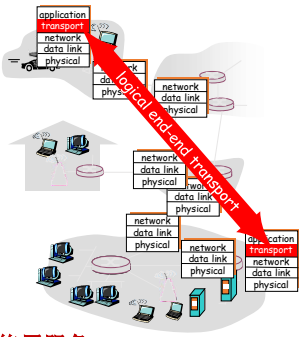


传输层和网络层关系：一个类比

- 设想A家庭有12个孩子，B家庭也有12个孩子，他们每周通过手写信件进行联系：
  - A家庭推选Ann，B家庭推选Bill，分别负责各自家庭的信件收发
  - Ann和Bill定期收集家庭中其他孩子的信件，投递到门口的邮筒里
  - Ann和Bill定期打开家庭邮箱，取出里面的信件，分发给其他孩子
  - 邮政系统负责将邮筒里的信件投递到收信人的家庭邮箱里
- 类比：
  - 进程 = 孩子
  - 进程间通信 = 孩子之间通信
  - 应用报文 = 信件
  - 传输层 = Ann 和 Bill（提供人到人的服务）
  - 终端 = 家庭住宅
  - 网络层 = 邮政系统（提供门到门的服务）

3. 传输层提供什么服务?

- 因特网的网络层提供**“尽力而为”**的服务：
  - 网络层尽最大努力在终端间交付分组，但不提供任何承诺
  - 具体来说，不保证交付，不保证按序交付，不保证数据完整，不保证延迟，不保证带宽等
- 传输层的**有所为**、**有所不为**:
  - 传输层可以通过差错恢复、重排序等手段提供可靠、按序的交付服务；也可以不提供
  - 传输层无法提供延迟保证、带宽保证等服务



传输层依赖网络层服务，并扩展网络层服务

因特网传输层提供的服务

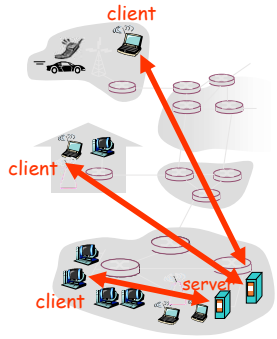
- 最低限度的传输服务：
  - 将终端-终端的数据交付扩展到进程-进程的数据交付（6.3节）
  - 报文检错
- 增强服务：
  - 可靠数据传输
  - 流量控制
  - 拥塞控制
- 因特网传输层通过UDP协议和TCP协议，向应用层提供两种不同的传输服务：
  - UDP协议（6.4节）：仅提供最低限度的传输服务
  - TCP协议（6.5节，6.6节）：提供基础服务和增强服务

本章内容

6.1 传输层服务（p3）
6.2 套接字（Socket）编程（p10）
6.3 传输层复用和分用(p37)
6.4 无连接传输：UDP (p47)
6.5 面向连接的传输：TCP (p55)
6.6 TCP拥塞控制(p100)
6.7 拥塞控制的发展
6.8 传输层协议的发展

1. 如何创建一个网络应用?

- 如何创建一个网络应用：
  - 编写一个分布式程序，使其运行在不同的端系统上，并通过网络通信
- 选择一种应用程序体系结构：
  - 传统及主流的是 客户-服务器体系结构（C/S）
  - 有一台总是在线的主机，运行一个服务器程序(server)，服务器主机具有永久、众所周知的地址
  - 用户终端上运行一个客户程序（client），需要时主动与服务器程序通信，请求服务
  - 客户只与服务器通信，客户之间不通信



2. 进程如何标识自己

- 每个进程需要一个标识，以便其它进程能够找到它
  - 问题：可以用进程所在主机的地址来标识进程吗?
  - 回答：不能！因为同一个主机上可能运行着许多进程
- 端口号（port number）：
  - 端口号用来区分同一个主机的、基于TCP/IP的不同进程
- 因此，进程标识包括：
  - 主机地址（IP地址）
  - 主机上与该进程关联的端口号（Port号）

注：

PID是操作系统内部的管理标识，而Port是网络通信的寻址标识，两者共同协作实现进程间的网络通信。

它们之间的关系取决于具体的应用程序和网络架构。

3. 应用编程接口：Socket API

- Socket API于1981年在BSD UNIX 4.1中引入，后成为因特网程序的标准编程接口，实现主机间进程通信。它是应用层与TCP/IP协议簇通信的中间抽象层。
- 它通过五元组（协议、源IP、源端口、远程IP、远程端口）唯一标识一对网络通信端点（链接）。
- 使用时需显式地创建、使用和释放套接字（socket）
- 采用C/S模式：客户主动发起通信请求，服务器始终等待客户请求到来
- 应用可以通过Socket API调用两种主要传输服务:
  - 不可靠的数据报服务：由UDP协议实现
  - 可靠的字节流服务：由TCP协议实现

本小节的编程实例：回音（echo）服务

1）客户程序从键盘读入一行字符（数据），发送给服务器

2）服务器接收数据，然后将收到的数据回送给客户

3）客户接收回送的数据，在屏幕上显示出来

参考实例：

https://blog.csdn.net/weixin_59093392/article/details/132738045

https://zhuanlan.zhihu.com/p/402313585

1）套接字地址：通用地址

- 套接字API接口的地址类型sockaddr，作为不同类型套接字地址的“虚父类”，在套接字API的定义；
- 在编程中大多数是使用sockaddr_in这个结构来设置/获取地址信息，调用时为防止编译错误需要做强制类型转换。

```
struct sockaddr{            /* struct to hold an address*/
   u_short sa_family;      /*type of address*/
   char    sa_data[14];    /*value of address*/
}
```

```
struct sockaddr_in {  //struct to hold an address
    sa_family_t sin_family;  //always AF_INET
    in_port_t sin_port;  //protocol port number: uint16_t
    struct in_addr sin_addr; //IP address
    char sin_zero[8];        //unused(set to zero)
};
```


套接字地址：TCP/IP地址

两个结构体长度都是16个字节，可互相转化。指向sockaddr_in结构的指针也可指向sockaddr。sockaddr常用于bind、connect、recvfrom、sendto等函数的参数，指明地址信息，是一种通用的套接字地址。

sockaddr_in 是Internet环境下套接字的地址形式。网络编程中会对sockaddr_in结构体进行操作。一般先把sockaddr_in变量赋值后，强制类型转换后传入用sockaddr做参数的函数。sockaddr_in用于socket定义和赋值；sockaddr用于函数参数。

- sa_family：2字节的地址家族，一般采用“AF_xxx”形式，常用地址类，
- sin_port：存放端口号(按照网络字节序存储)
- sin_addr：存放32位IP地址(无符号整数)

```
struct in_addr {
    in_addr_t s_addr;   //IPv4 address (uint32_t)};
```

- sin_zero：为与sockaddr大小兼容而保留的空字节

2）创建套接字：socket()

- 客户或服务器调用socket( )创建本地套接字，返回套接字描述符
- domain指明网络层地址类型
- type指明传输层协议：
  - SOCK_STREAM代表TCP字节流
  - RAW_Socket（原始套接字-网络层）
  - SOCK_DGRAM代表UDP数据报

```
#include <sys/socket.h>

int socket(
    int domain, /* AF_UNIX, AF_INET, etc. */
    int type,  /* SOCK_STREAM, SOCK_DGRAM */
    int protocol); /* usually zero */
 /* Returns file descriptor or -1 on error (sets errno) */
```


套接字描述符

- Unix系统中程序通过访问套接字描述符socket_fd来进行套接字通信
- 套接字描述符socket_fd在类型上与文件描述符fd完全兼容
- 在套接字描述符上的发送、接收数据的行为可以用文件读写调用完成
  - read从描述符fd中读取(接收)长度为返回值的字节数至buf中
  - write向描述符fd中写入(发送)buf中长度为返回值的字节数

```
#include <unistd.h>

int read(int fd, void *buf, size_t count);
int write(int fd, void *buf, size_t count);
```


3）绑定套接字地址：bind()

- 用来将服务器本地套接字地址sockaddr与描述符socket_fd绑定
- 客户程序不需要调用bind()，操作系统将为其在1024～5000之间分配一个端口号

```
#include <sys/socket.h>

int bind(
    int socket_fd,
    const struct sockaddr *sa,
    socklen_t sa_len);
 /* Returns 0 if OK or -1 on error (sets errno) */
```


4）关闭套接字：close()

- 客户和服务器调用close()关闭一个套接字
- 当实参fd为TCP套接字描述符时：
  - 调用close()会引起本地进程向远程进程发送关闭连接的消息
- 当实参fd为UDP套接字描述符时：
  - 调用close()会引起为此描述符分配的资源被内核释放

```
#include <unistd.h>
int close(int fd);
 /* Returns 0 if OK or -1 on error */
```

实例1：使用UDP套接字实现回音服务

➢客户或服务器调用sendto()发送数据：

- 数据在buff中，长度为nbytes字节
- 对方通信进程的地址在to中

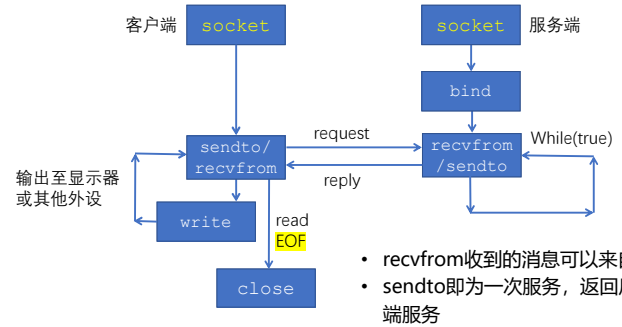
```
#include <sys/socket.h>
ssize_t sendto(
        int socket_fd,
        const void *buff,
        size_t nbytes,
        int flags, /* usually 0 */
        const struct sockaddr *to,
        socklen_t *addrlen,
        );
/*Return number of bytes written
if OK or -1 on error*/
```

➢客户或服务器调用recvfrom()接收数据：

- 收到的数据放在buff中
- 对方通信进程的地址在from中

```
#include <sys/socket.h>
ssize_t recvfrom(
        int socket_fd,
        void *buff,
        size_t nbytes,
        int flags, /* usually 0*/
        struct sockaddr *from,
        socklen_t *addrlen,
        );
/*Return number of bytes read if
OK or -1 on error*/
```


基于UDP的套接字通信流程



- recvfrom收到的消息可以来自不同的客户端；
- sendto即为一次服务，返回后，继续为下个客户端服务

UDP回音服务端代码：main

```
1  int main(int argc, char **argv){
2     char mesg[MAXLINE];
3     int sockfd, n, len;
4     struct sockaddr_in cliaddr, servaddr;
5
6     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
7     bzero(&servaddr, sizeof(servaddr));
8     servaddr.sin_family = AF_INET;
9     servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
10    servaddr.sin_port = htons(SERV_PORT);
11
12    bind(sockfd, (struct sockaddr *) &servaddr, sizeof(servaddr));
13
14    for ( ; ; ) {
15       len = sizeof(cliaddr);
16       n = recvfrom(sockfd, mesg, MAXLINE, 0, cliaddr, &len);
17        sendto(sockfd, mesg, n, 0, cliaddr, len);
18       }
19 }
```


UDP回音客户端代码：Part 1

```
1  int main(int argc, char **argv){
2    int sockfd, n;
3    struct sockaddr_in servaddr;
4    char sendline[MAXLINE], recvline[MAXLINE + 1];
5    if (argc != 2){
6       fprintf(stderr, %s\n, "usage: echoCli <IPaddress>");
7       exit(EXIT_FAILURE);
8     }
9    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
10    bzero(&servaddr, sizeof(servaddr));
11   servaddr.sin_family = AF_INET;
12   inet_pton(AF_INET, argv[1], &servaddr.sin_addr);
13   servaddr.sin_port = htons(SERV_PORT);
```

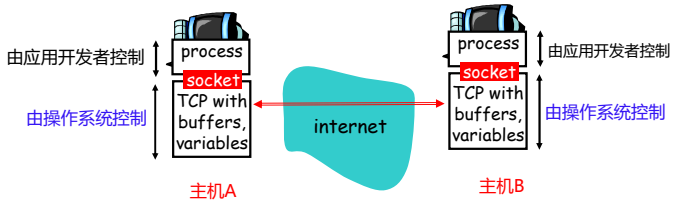
UDP回音客户端代码：Part 2

```
14  while (fgets(sendline, MAXLINE, stdin) != NULL) {
15     sendto(sockfd, sendline, strlen(sendline), 0,
                   (struct sockaddr *)servaddr, sizeof(servaddr));
16     n = recvfrom(sockfd, recvline, MAXLINE, 0, NULL, NULL);
17     recvline[n] = 0;
18     fputs(recvline, stdout);
19    }
20   exit(0);
21 }
```


实例2：使用TCP套接字实现回音服务

➢使用TCP套接字编程时，可将TCP连接想像成一对套接字之间的一条封闭管道：

- 发送端TCP将要发送的字节序列从管道的一端（套接字）送入
- 接收端TCP从管道的另一端（套接字）取出字节序列
- 在管道中传输的字节不丢失，并保持顺序



服务器使用多个套接字服务客户

➢服务器进程在监听套接字上等待客户的连接请求

➢客户进程创建一个本地套接字，与服务器的监听套接字通信：

- 在此过程中，客户TCP向服务器TCP发送连接请求

➢收到连接请求后，服务器进程创建一个临时套接字（称连接套接字）和一个新的服务器进程，与客户进程通信

➢服务器进程回到监听套接字上继续等待：

- 此举允许服务器同时服务多个客户

➢客户服务结束后，服务器销毁进程，关闭连接套接字

5）TCP服务器监听套接字：listen()

➢将主动套接字转变为被动套接字，进入监听状态；

➢backlog设定了监听队列长度

```
#include <sys/socket.h>

int listen(
    int socket_fd;
    int backlog;) //maximum connection queue length
 /* Returns 0 if OK or -1 on error */
```

6）TCP套接字系统调用：accept()

- 服务端从已建立连接的队列中取出队列一个客户端
- 返回值是用来和客户端交换数据的连接套接字描述符，需要在通信完毕后调用close关闭
- 返回时客户端的IP地址和端口可在cliaddr中访问

```
#include <sys/socket.h>

int accept(
    int socket_fd,
    struct sockaddr *cliaddr,  /* client address */
    socklen_t *addrlen); /* address struct length */
 /* Returns file descriptor or -1 on error */
```


6）TCP套接字系统调用：connect()

- 客户端调用connect向服务器发起建立连接的请求
- 服务器的IP地址和端口信息在servaddr中
- 三次握手的过程导致connect的调用是阻塞的

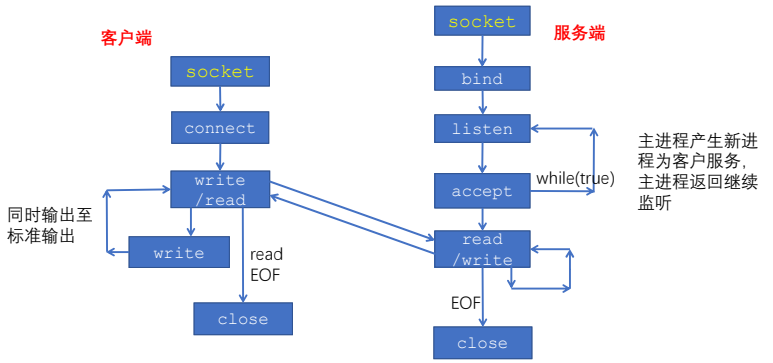
```
#include <sys/socket.h>

int connect(
    int socket_fd,
    const struct sockaddr *servaddr, /* server address */
    socklen_t *addrlen); /* address struct length */
 /* Returns 0 or -1 on error */
```


回音服务

- 服务端在SERV_PORT（9999）监听
- 客户端在终端下运行命令
  - #echoCli <Server IPaddress>
  - c: Hello
  - s: Hello
  - c: Bye
  - s: Bye
  - c: Ctrl+D
  - #

基于TCP的套接字通信流程



TCP回音服务端代码：main

```
1  int main(int argc, char **argv){
2     int listenfd, connfd;
3     socklen_t clilen;
4     struct sockaddr_in cliaddr, servaddr;
5
6     listenfd = socket(AF_INET, SOCK_STREAM, 0);
7     bzero(&servaddr, sizeof(servaddr));
8     servaddr.sin_family = AF_INET;
9     servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
10    servaddr.sin_port = htons(SERV_PORT);
11    bind(listenfd, (struct sockaddr *) &servaddr, sizeof(servaddr));
12    listen(listenfd, LISTENQ);
13
14    for ( ; ; ) {
15       clilen = sizeof(cliaddr);
16       connfd = accept(listenfd, (struct sockaddr *) &cliaddr, &clilen);
17       str_echo(connfd); /* process the request */
18       }
19 }
```


服务端通过sockfd提供回音服务

```
1  void str_echo(int sockfd)
2  {
3     ssize_t     n;
4     char        buf[MAXLINE];
5
6  again:
7      while ( (n = read(sockfd, buf, MAXLINE)) > 0)
8          write(sockfd, buf, n);
9      if (n < 0 && errno == EINTR)
10        goto again; //soft interrupt, try again
11     else if (n < 0){
12         perror("fail to read");
13         exit(EXIT_FAILURE);
14 }
```


回音客户端代码：main

```
1  int main(int argc, char **argv){
2     int sockfd;
3     struct sockaddr_in servaddr;
4
5     if (argc != 2){
6           fprintf(stderr, %s\n, "usage: echoCli <IPaddress>");
7           exit(EXIT_FAILURE);
8     }
9     sockfd = socket(AF_INET, SOCK_STREAM, 0);
10
11    bzero(&servaddr, sizeof(servaddr));
12    servaddr.sin_family = AF_INET;
13    servaddr.sin_port = htons(SERV_PORT);
14    inet_pton(AF_INET, argv[1], &servaddr.sin_addr);
15    connect(sockfd, (struct sockaddr *) &servaddr, sizeof(servaddr));
16    str_cli(stdin, sockfd);
17    exit(EXIT_SUCCESSFUL);
18 }
```


客户端通过sockfd获得回音服务

```
1  void str_cli(FILE *fp, int sockfd)
2  {
3      char sendline[MAXLINE], recvline[MAXLINE];
4
5      while (fgets(sendline, MAXLINE, fp) != NULL) {
6          write(sockfd, sendline, strlen(sendline));
7
8          if (read(sockfd, recvline, MAXLINE) == 0){
9              fprintf(stderr, %s\n, "server terminated prematurely");
10             exit(EXIT_FAILURE);
11         }
12         fputs(recvline, stdout);
13     }
14 }
```

本章内容

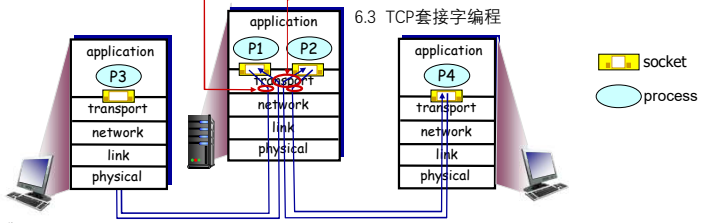
6.1 传输层服务
6.2 套接字Socket编程
6.3 传输层复用和分用
6.4 无连接传输：UDP
6.5 面向连接的传输：TCP
6.6 TCP拥塞控制
6.7 拥塞控制的发展
6.8 传输层协议的发展

传输层基本服务

传输层基本服务：将主机间交付扩展到进程间交付，通过复用和分用实现

（发送端）复用:
传输层从多个套接字收集数据，交给网络层发送

（接收端）分用:
传输层将从网络层收到的数据，交付给正确的套接字



如何进行复用和分用：类比

➢为将邮件交付给收信人：
- 每个收信人应有一个信箱，写有收信人地址和姓名（唯一标识）
- 信封上有收信人的地址和名字

➢为将报文段交付给套接字：
- 主机中每个套接字应分配一个唯一的标识
- 报文段中包含接收套接字的标识

➢复用：
- 发送方传输层将套接字标识置于报文段中，交给网络层

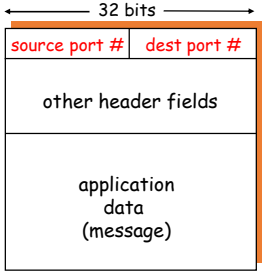
➢分用：
- 接收方传输层根据报文段中的套接字标识，将报文段交付到正确的套接字

套接字标识与端口号

➢端口号是套接字标识的一部分：
- 每个套接字在本地关联一个端口号
- 端口号是一个16比特的数

➢端口号的分类：
- 熟知端口：0～1023，由公共域协议使用
- 注册端口：1024～49151，需要向IANA注册才能使用
- 动态和/或私有端口：49152～65535，一般程序使用

➢报文段中有两个字段携带端口号
- 源端口号：与发送进程关联的本地端口号
- 目的端口号：与接收进程关联的本地端口号



TCP/UDP报文段格式

套接字端口号的分配

➢自动分配：

- 创建套接字时不指定端口号
- 由操作系统从49152～65535中分配
- 客户端通常使用这种方法

➢使用指定端口号创建套接字：

- 创建套接字时指定端口号
- 实现公共域协议的服务器应分配众所周知的端口号（0～1023）
- 服务器通常采用这种方法

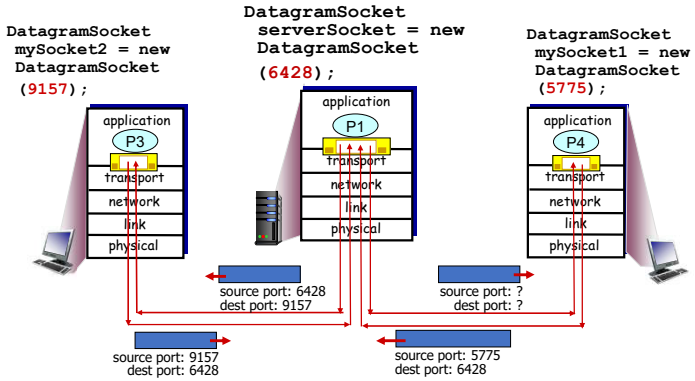
1. UDP分用的方法

➢UDP套接字使用<IP地址, 端口号>二元组进行标识

➢接收方传输层收到一个UDP报文段后：

- 检查报文段中的目的端口号，将UDP报文段交付到具有该端口号的套接字
- <目的IP地址，目的端口号> 相同的UDP报文段被交付给同一个套接字，与 <源IP地址，源端口号> 无关
- 报文段中 <源IP地址，源端口号> 被接收进程用来发送响应报文

UDP分用: 举例



2. TCP服务器使用的套接字

➢一个TCP服务器为了同时服务很多个客户，使用两种套接字

➢监听套接字：

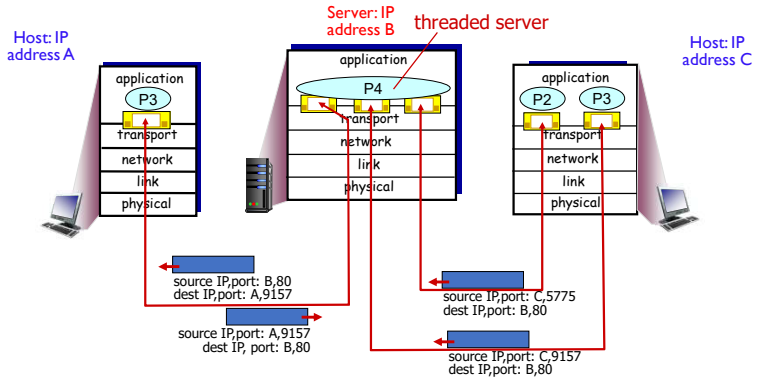
- 服务器平时在监听套接字上等待客户的连接请求，该套接字具有众所周知的端口号

➢连接套接字：

- 服务器在收到客户的连接请求后，创建一个连接套接字，使用临时分配的端口号
- 服务器同时创建一个新的进程，在该连接套接字上服务该客户
- **每个连接套接字只与一个客户通信**，即只接收具有以下四元组的报文段：
  - 源IP地址 = 客户IP地址，源端口号 = 客户套接字端口号
  - 目的IP地址 = 服务器IP地址，目的端口号 = 服务器监听套接字的端口号

➢**连接套接字需要使用上述<源IP地址，源端口号，目的IP地址，目的端口号>四元组进行标识**，服务器使用该四元组将TCP报文段交付到正确的连接套接字

TCP分用: 举例



小结:

➢UDP套接字

- 使用<IP地址，端口号>二元组标识UDP套接字
- 服务器使用一个套接字服务所有客户

➢TCP套接字

- 使用<源IP地址，目的IP地址，源端口号，目的端口号> 四元组标识连接套接字
- 服务器使用一个监听套接字和多个连接套接字服务多个客户，每个连接套接字服务一个客户

本章内容

6.1 传输层服务
6.2 套接字Socket编程
6.3 传输层复用和分用
6.4 无连接传输：UDP
6.5 面向连接的传输：TCP
6.6 TCP拥塞控制
6.7 拥塞控制的发展
6.8 传输层协议的发展

UDP提供的服务

➢网络层协议提供的服务（Best-effort service）：

- 尽最大努力将数据包交付到目的主机
- 不保证投递的可靠性和顺序
- 不保证带宽及延迟要求

➢UDP提供的服务：

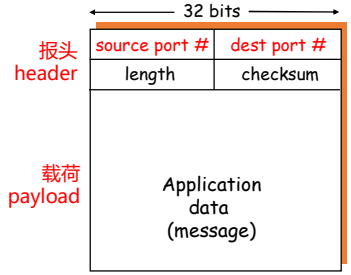
- 进程到进程之间的报文交付
- 报文完整性检查（可选）：检测并丢弃出错的报文

➢UDP需要实现的功能：

- 复用和分用
- 报文检错

UDP报文段结构

- UDP报文：
  - 报头(header)：携带协议处理需要的信息
  - 载荷(payload)：携带上层数据
- 用于复用和分用的字段：
  - 源端口号(source port)
  - 目的端口号(dest port)
- 用于检测报文错误的字段：
  - 报文总长度(length)：字节数
  - 校验和(checksum)：16bits（3部分：伪头，UDP头，载荷）



UDP报文格式

UDP校验和（checksum）

校验和(checksum)字段的作用：对传输的报文段进行检错

发送方：

- 将报文段看成是由16比特整数组成的序列
- 对这些整数序列计算校验和
- 将校验和放到UDP报文段的checksum字段

接收方：

- 对收到的报文段进行相同的计算
- 与报文段中的checksum字段进行比较：
  - 不相等：说明报文段有错误
  - 相等：认为报文段没有错误

校验和计算：举例

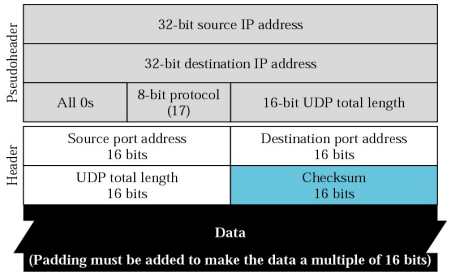
例子: add two 16-bit integers

```
                1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0
                1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
              ---------------------------------
wraparound  (1) 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1
              ---------------------------------
sum             1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0
checksum        0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1
```

Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

UDP校验和计算

- 计算UDP校验和时，要包括伪头、UDP头和数据三个部分
- UDP伪头信息取自IP报头，包括：
  - 源IP地址，目的IP地址
  - UDP的协议号
  - UDP报文段总长度
- 计算校验和时包含伪头信息，是为了避免由于IP地址错误等造成的误投递
- UDP校验和的使用是可选的，若不计算校验和，该字段填入0



UDP校验和计算举例

➢发送方计算校验和时，checksum字段先填0；

➢接收方对UDP报文（含校验和）及伪头求和，若结果为0xFFFF，认为无错

153.18.8.105			
171.2.14.10			
All 0s	17	15	
1087		13	
15		All 0s	
T	E	S	T
I	N	G	All 0s

```
10011001 00010010 ──→ 153.18
00001000 01101001 ──→ 8.105
10101011 00000010 ──→ 171.2
00001110 00001010 ──→ 14.10
00000000 00010001 ──→ 0 and 17
00000000 00001111 ──→ 15
00000100 00111111 ──→ 1087
00000000 00001101 ──→ 13
00000000 00001111 ──→ 15
00000000 00000000 ──→ 0 (checksum)
01010100 01000101 ──→ T and E
01010011 01010100 ──→ S and T
01001001 01001110 ──→ I and N
01000111 00000000 ──→ G and 0 (padding)
10010110 11101011 ──→ Sum
01101001 00010100 ──→ Checksum
11111111 11111111 ──→ FFFF
```


为什么需要UDP?

为什么需要UDP?

➢应用可以尽可能快地发送报文：
- 无建立连接的延迟
- 不限制发送速率（不进行拥塞控制和流量控制）

➢报头开销小

➢协议处理简单

UDP适合哪些应用?

➢容忍丢包但对延迟敏感的应用：
- 如流媒体

➢以单次请求/响应为主的应用：
- 如DNS，SNMP

➢若应用要求基于UDP进行可靠传输：
- 可由应用层实现可靠性

本章内容

6.1 传输层服务
6.2 套接字Socket编程
6.3 传输层复用和分用
6.4 无连接传输：UDP
6.5 面向连接的传输：TCP
6.6 TCP拥塞控制
6.7 拥塞控制的发展
6.8 传输层协议的发展

1. TCP概述
2. TCP报文段结构
3. TCP可靠数据传输
4. TCP流量控制
5. TCP连接管理

https://mp.weixin.qq.com/s/8xdO94JFGHP_tWg_v_u75w

1. TCP 概述

➢TCP服务模型：
- 在一对通信的进程之间提供一条理想的字节流管道

➢点到点通信:
- 仅涉及一对通信进程 （一个连接）

➢全双工：
- 可以同时双向传输数据

➢可靠、有序的字节流：
- 不保留报文边界

需要的机制

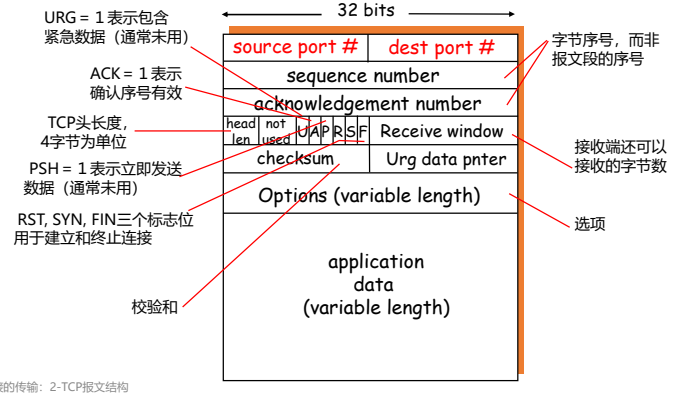
➢建立连接:
- 通信双方为本次通信建立数据传输所需的状态（套接字、缓存、变量等）

➢可靠数据传输：
- 流水线式发送，报文段检错，丢失重传

➢流量控制:
- 发送方不会令接收方缓存溢出



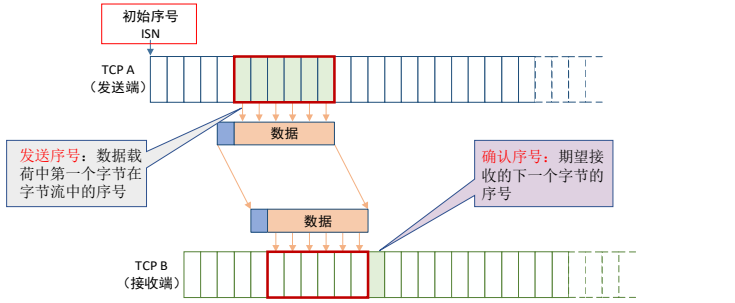
2. TCP报文段结构



重要的TCP选项

- 最大段长度（Max Segment Size）：
  - TCP段中可以携带的最大数据字节数
  - 建立连接时，每个主机可声明自己能够接收的MSS，缺省为536字节
- 窗口比例因子（window scale）：
  - 建立连接时，双方可以协商一个窗口比例因子
  - 实际接收窗口大小 = window size × 2 window scale
- 选择确认（SACK）：
  - 最初的TCP协议只使用累积确认
  - 改进的TCP协议引入选择确认，允许接收端指出缺失的数据字节

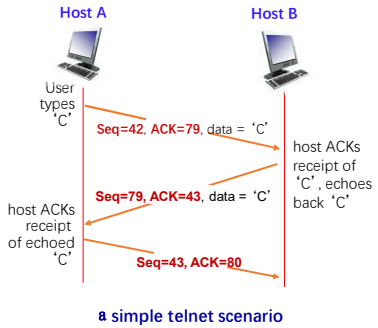
发送序号和确认序号的含义



初始序号的选取：每个TCP实体维护一个32位计数器，该计数器每4微秒增1，建立连接时从中读取计数器当前值（依赖具体实现，见连接管理）

发送序号和确认序号：举例

- 主机A向主机B发送仅包含一个字符‘C’的报文段：
  - 发送序号为42
  - 确认序号为79（对前一次数据的确认）
- 主机B将字符‘C’回送给主机A：
  - 发送序号为79
  - 确认序号为43（对收到‘C’的确认）
- 主机A向主机B发送确认报文段（不包含数据）：
  - 发送序号43（浪费）
  - 确认序号为80（对收到‘C’的确认）



3. TCP可靠数据传输

- TCP 在不可靠的IP服务上建立可靠的数据传输
- 基本机制
  - 发送端：流水线式发送数据、等待确认、超时重传
  - 接收端：进行差错检测，采用累积确认机制
- 乱序段处理：协议没有明确规定
  - 接收端不缓存：可以正常工作，处理简单，但效率低
  - 接收端缓存：效率高，但处理复杂

实例：一个简化的TCP协议

简化的TCP协议：仅考虑可靠传输机制，且数据仅在一个方向上传输。设定：

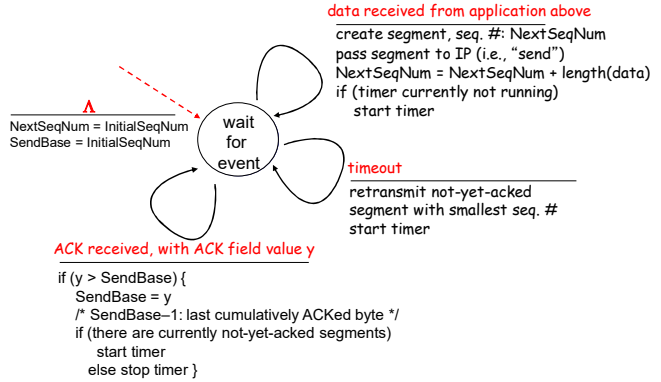
- 发送方：
  - 发送策略：流水线式发送报文段
  - 定时器的使用：仅对最早未确认的报文段使用一个重传定时器（与GBN类似）
  - 重发策略：仅在超时后重发最早未确认的报文段（与SR类似）
- 接收方：
  - 确认方式：采用累积确认，仅在正确、按序收到报文段后，更新确认序号；其余情况，重复前一次的确认序号（与GBN类似）
  - 失序报文段处理：缓存失序的报文段（有较大缓存，类似SR）

1）TCP发送方要处理的事件

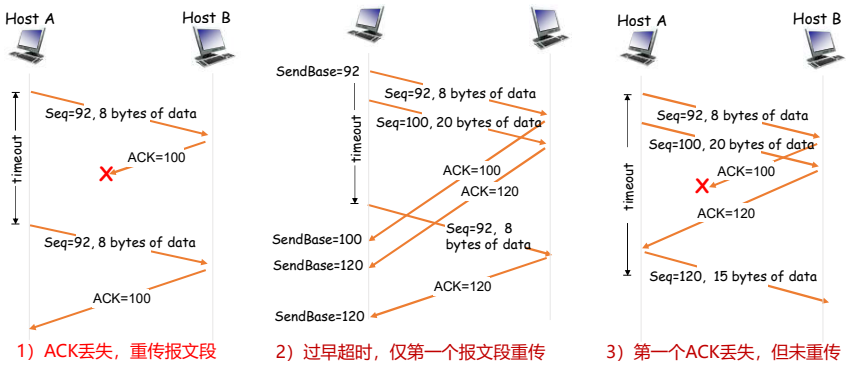
发送方：

- 收到应用数据（data received from application above）
  - 创建并发送TCP报文段
  - 若当前没有定时器在运行（没有已发送、未确认的报文段），启动定时器
- 定时器超时（timeout）
  - 重传包含最小序号的、未确认的报文段
  - 重启定时器
- 收到ACK（ACK received, with ACK field value y ）
  - 如果确认序号大于基序号（已发送未确认的最小序号）：
    - 滑动发送窗口（更新基序号）
    - 如果发送窗口中还有未确认的报文段，启动定时器；否则终止定时器

TCP发送端状态机（简化版）



TCP可能的重传场景



1）ACK丢失，重传报文段　2）过早超时，仅第一个报文段重传　3）第一个ACK丢失，但未重传

TCP可靠数据传输概述

➢思考以下问题：
- 第二种情形，如果TCP像SR一样，每个报文段使用一个定时器，会怎么样？
- 第三种情形，采用流水式发送和累积确认，可避免重发哪些报文段？

➢可见，TCP通过采用以下机制减少了不必要的重传：
- 发送方只使用一个定时器，避免了超时设置过小时重发大量报文段（第2种情形）；若设置过大，会造成带宽浪费
- 利用流水式发送和累积确认，可避免重发某些丢失了ACK的报文段（第3种情形）

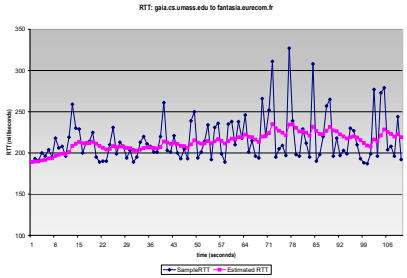
➢想一想：定时器超时值设置多大比较合适呢？

2）如何设置定时器超时值？

➢为什么设置合理的超时值很重要：
- 若超时值太小，容易产生不必要的重传
- 若超时值太大，则丢包恢复的时间太长

➢直观上超时值应大于RTT，但RTT是变化

➢如何估计RTT：
- RTT是变化的，需要实时测量从发出某个报文段到收到其确认报文段之间经过的时间（SampleRTT，瞬时RTT）
- 由于SampleRTT波动大，更有意义的是估算其平均值（EstimatedRTT，平均RTT）



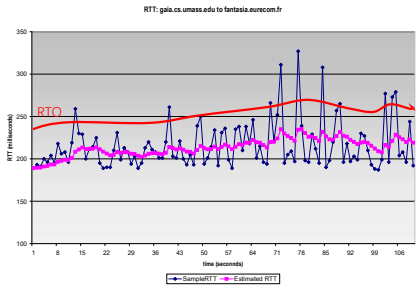
★平均RTT的估算方法（指数加权移动平均）：
- EstimatedRTT = (1- α)*EstimatedRTT + α*SampleRTT (典型地，α=0.125)

如何设置定时器超时值？

➢瞬时RTT和平均RTT有很大的偏差：
- 需要在EstimtedRTT上加一个“安全距离”，作为超时值
- 安全距离的大小与RTT的波动幅度有关

➢估算SampleRTT 与 EstimatedRTT的偏差（称DevRTT）：
- DevRTT = (1-β)*DevRTT + β*|SampleRTT-EstimatedRTT|

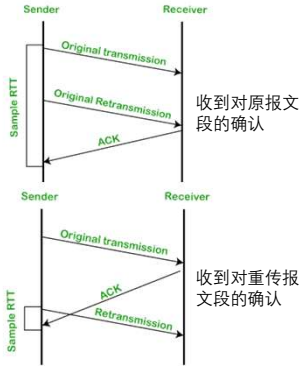
典型地，β = 0.25



➢设置重传定时器的超时值RTO：
- RTO = EstimatedRTT + 4*DevRTT

TCP确认的二义性对定时器的影响

- TCP确认的二义性问题：
  - 重传的TCP报文段使用与原报文段相同的序号
  - 发送端收到确认后，无法得知是对哪个报文段进行的确认
- 二义性确认带来的问题：
  - 对重传报文段测量的SampleRTT，可能不准确
- 解决方法：
  - 忽略有二义性的确认，只对一次发送成功的报文段测量SampleRTT，并更新EstimtedRTT
  - 当TCP重传一个段时，停止测量SampleRTT



定时器补偿

- 简单忽略重传报文段的问题：
  - 重传意味着超时值可能偏小了，需要增大
  - 若简单忽略重传报文段（不更新Estimted RTT），则超时值也不会更新，超时设置过小的问题没有解决
- 解决方法：
  - 采用定时器补偿策略，发送方每重传一个报文段，就直接将超时值增大一倍（不依赖于RTT的更新）
  - 若连续发生超时事件，超时值呈指数增长（至一个设定的上限值）

Karn算法

- Karn算法结合使用RTT估计值和定时器补偿策略确定超时值：
  - 使用Estimated RTT估计初始的超时值
  - 若发生超时，每次重传时对定时器进行补偿，直到成功传输一个报文段为止
  - 若收到上层应用数据、或某个报文段没有重传就被确认了，用最近的Estimated RTT估计超时值。

3）TCP的接收端

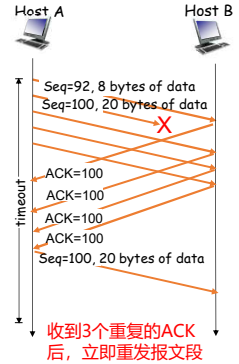
- 理论上，接收端只需区分两种情况：
  - 收到期待的报文段：发送更新的确认序号
  - 其它情况：重复当前的确认序号
- 为减小通信量，TCP允许接收端推迟确认：
  - 接收端可以在收到若干个报文段后，发送一个累积确认的报文段
- 推迟确认带来的问题：
  - 若延迟太大，会导致不必要的重传
  - 推迟确认造成RTT估计不准确
- 因此，TCP协议规定：
  - 推迟确认的时间最多为500ms
  - 接收方至少每隔一个报文段使用正常方式进行确认

TCP接收端的事件和处理

接收端事件	接收端动作
收到一个期待的报文段，且之前的报文段均已发送过确认	推迟发送确认，在500ms时间内若无下一个报文段到来，发送确认
收到一个期待的报文段，且前一个报文段被推迟确认	立即发送确认（估计RTT的需要）
收到一个失序的报文段（序号大于期待的序号），检测到序号间隙	立即发送确认（快速重传的需要），重复当前的确认序号
收到部分或全部填充间隙的报文段	若报文段始于间隙的低端，立即发送确认（推进发送窗口），更新确认序号

4）快速重传

- 仅靠超时重发丢失的报文段，恢复太慢！
- 发送方可利用重复ACK检测报文段丢失：
  - 发送方通常连续发送许多报文段
  - 若仅有个别报文段丢失，发送方将收到多个重复序号ACK
  - 多数情况下IP按序交付分组，重复ACK有可能因丢包产生
- TCP协议规定：
  - 当发送方收到对同一序号的3次重复确认时，立即重发包含该序号的报文段
- 快速重传：
  - 所谓快速重传，就是在定时器到期前重发丢失的报文段



快速重传算法

```
event: ACK received, with ACK field value of y
            if (y > SendBase) { //收到更新的确认号
                SendBase = y
                if (there are currently not-yet-acknowledged segments)
                    start timer
              }
            else { //收到重复序号的ACK
                increment count of dup ACKs received for y
                if (count of dup ACKs received for y = 3) {
                    resend segment with sequence number y    //快速重传
                }
```


TCP可靠传输---小结

- TCP可靠传输的设计要点：
  - 流水式发送报文段
  - 缓存失序的报文段
  - 采用累积确认
  - 只对最早未确认的报文段使用一个重传定时器
  - 超时后只重传包含最小序号的、未确认的报文段
- 以上措施可大量减少因ACK丢失、定时器过早超时引起的重传
- 超时值的确定：
  - 基于RTT估计超时值 + 定时器补偿策略
- 测量RTT：
  - 不对重传的报文段测量RTT
  - 不连续使用推迟确认
- 快速重传：
  - 收到3次重复确认ACK，立即重发报文段

问：TCP使用GBN还是SR?

Go-Back-N

➢接收方：
- 使用累积确认
- 不缓存失序的分组
- 对失序分组发送重复ACK

➢发送方：
- 超时后重传从基序号开始的所有分组

TCP

➢接收方：
- 使用累积确认
- 缓存失序的报文段
- 对失序报文段发送重复ACK
- 增加了推迟确认

➢发送方：
- 超时后仅重传最早未确认的报文段
- 增加了快速重传

问：TCP使用GBN还是SR?

SR

➢接收方：
- 缓存失序的分组
- 单独确认每个正确收到的分组

➢发送方：
- 每个分组使用一个定时器
- 仅重传未被确认的分组

修改的TCP [RFC2018]

➢接收方：
- 缓存失序的报文段
- SACK选项头中给出非连续数据块的上下边界

➢发送方：
- 只对最早未确认的报文段使用一个定时器
- 仅重传接收方缺失的数据
- 增加了快速重传

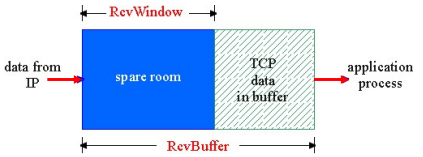
TCP结合了GBN和SR的优点

➢TCP的差错恢复机制可以看成是GBN和SR的混合体：
- 定时器的使用：与GBN类似，只对最早未确认的报文段使用一个定时器
- 超时重传：与SR类似，只重传缺失的数据

➢TCP在减小定时器开销和重传开销方面要优于GBN 和SR！

4. TCP的流量控制：接收缓存

➢TCP接收端有一个接收缓存：
- 接收端TCP将收到的数据放入接收缓存
- 应用进程从接收缓存中读数据
- 进入接收缓存的数据不一定被立即取走、取完
- 如果接收缓存中的数据未及时取走，后续到达的数据可能会因缓存溢出而丢失



➢流量控制：
- 发送端TCP通过调节发送速率，不使接收端缓存溢出

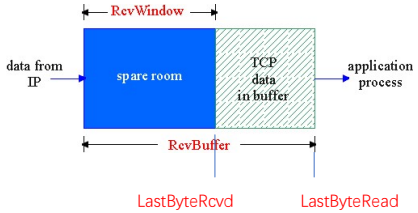
TCP如何进行流量控制

- 接收缓存中的可用空间称为接收窗口：

  RcvWindow = RcvBuffer-[LastByteRcvd - LastByteRead]

- 接收方将RcvWindow放在报头中，向发送方通告接收缓存的可用空间
- 发送方限制未确认的字节数不超过接收窗口的大小，即：

  LastByteSent-LastByteAcked ≦ RcvWindow



- 特别是，当接收方通告接收窗口为0时，发送方必须停止发送。

1）非零窗口通告

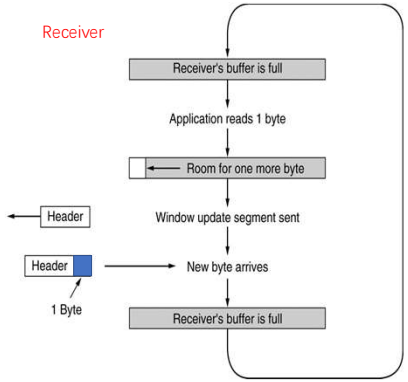
- 发送方/接收方对零窗口的处理：
  - 发送方：当接收窗口为0时，发送方必须停止发送
  - 接收方：当接收窗口变为非0时，接收方应通告增大的接收窗口
- 在TCP协议中，触发一次TCP传输需要满足以下三个条件之一：
  - 应用程序调用
  - 超时
  - 收到数据/确认
- 对于单向传输中的接收方，只有第三个条件能触发传输
- 当发送方停止发送后，接收方不再收到数据，如何触发接收端发送“非零窗口通告”呢？
- TCP协议规定：
  - 发送方收到“零窗口通告”后，可以发送“零窗口探测”报文段
  - 从而接收方可发送包含接收窗口的响应报文段。

零窗口探测的实现

- 发送端收到零窗口通告时，启动一个坚持定时器
- 定时器超时后，发送端发送一个零窗口探测报文段（序号为上一个段中最后一个字节的序号）
- 接收端在响应的报文段中通告当前接收窗口的大小
- 若发送端仍收到零窗口通告，重新启动坚持定时器
- 否则，恢复发送

2）糊涂窗口综合症

- 当数据的发送速度很快、而消费速度很慢时，零窗口探测的简单实现带来以下问题：
  - 接收方不断发送微小窗口通告
  - 发送方不断发送很小的数据分组
  - 大量带宽被浪费
- 解决方案：
  1）接收方启发式策略
  2）发送方启发式策略



①接收方启发式策略

➢接收端避免糊涂窗口综合症的策略：
- 通告零窗口之后，仅当窗口大小显著增加之后才发送更新的窗口通告
- 什么是显著增加？窗口大小达到缓存空间的一半或者一个MSS，取两者的较小值

➢TCP执行该策略的做法：
- 当窗口大小不满足以上策略时，推迟发送确认（但最多推迟500ms，且至少每隔一个报文段使用正常方式进行确认），寄希望于推迟间隔内有更多数据被消费
- 仅当窗口大小满足以上策略时，才通告新的窗口大小

② 发送方启发式策略

➢发送方避免糊涂窗口综合症的策略：
- 发送方应积聚足够多的数据再发送，以防止发送太短的报文段

➢问题：发送方应等待多长时间？
- 若等待时间不够，报文段会太短
- 若等待时间过久，应用程序的时延会太长
- 更重要的是，TCP不知道应用程序会不会在最近的将来生成更多的数据

➢Nagle算法的解决方法：
- 在新建连接上，当应用数据到来时，组成一个TCP段发送（那怕只有一个字节）
- 在收到确认之前，后续到来的数据放在发送缓存中
- 当数据量达到一个MSS或上一次传输的确认到来（取两者的较小时间），用一个TCP段将缓存的字节全部发走

➢Nagle算法的优点：

1）适应网络延时、MSS长度及应用速度的各种组合场景

2）常规情况下不会降低网络的吞吐量

TCP流量控制小结：

➢TCP接收端
- 使用显式的窗口通告，告知发送方可用的缓存空间大小；
- 在接收窗口较小时，推迟发送确认；
- 仅当接收窗口显著增加时，通告新的窗口大小。

➢TCP发送端
- 使用Nagle算法确定发送时机；
- 使用接收窗口限制发送的数据量，已发送未确认的字节数不超过接收窗口的大小。

问：为什么GBN/SR和UDP不需要流量控制

➢GBN和SR均假设：
- 正确、按序到达的分组被立即交付给上层，其占用的缓冲区被立即释放

➢发送方根据确认序号即可知道：
- 哪些分组已被移出接收窗口
- 接收窗口还可以接受多少分组

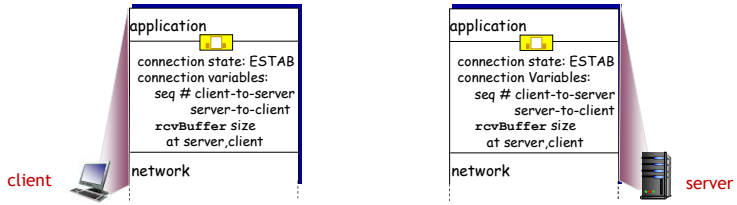
➢因此，GBN和SR本身就带有流量控制功能。

➢UDP不保证交付：
- 接收端UDP将收到的报文载荷放入接收缓存
- 应用进程每次从接收缓存中读取一个完整的报文载荷
- 当应用进程消费数据不够快时，接收缓存溢出，报文数据丢失，UDP不负责任。

➢因此，UDP不需要流量控制

5. TCP连接管理

- 建立一条TCP连接需要确定两件事：
  - 双方都同意建立连接（知晓另一方想建立连接）
  - 初始化连接参数（序号，MSS等）

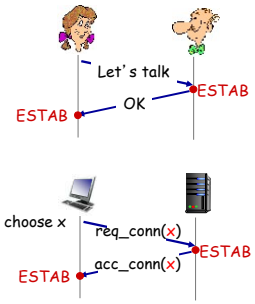



```
Socket clientSocket =
  newSocket("hostname","port
  number");
```

```
Socket connectionSocket =
  welcomeSocket.accept();
```

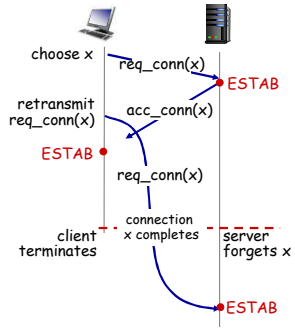

两次握手建立连接

两次握手建立连接的类比

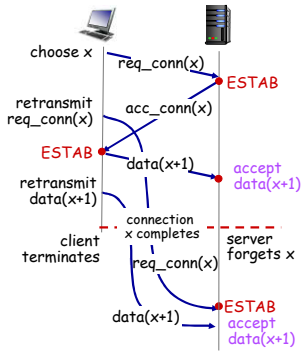


- 问题：
  - 在网络中，2次握手总是可行的吗？
- 在一个不可靠的网络中，总会有一些意外发生：
  - 包传输延迟变化很大
  - 存在重传的报文段
  - 存在报文重排序

两次握手失败的情形



形成半开连接（没有对应client）



形成半开连接（没有client）

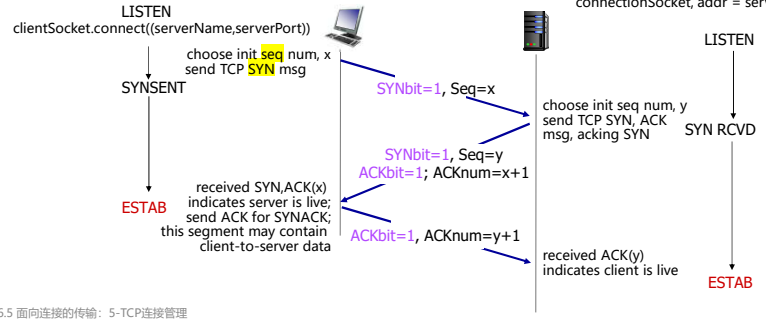
TCP三次握手建立连接

客户状态

```
clientSocket = socket(AF_INET, SOCK_STREAM)
LISTEN
clientSocket.connect((serverName,serverPort))
```

服务器状态

```
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(( " ,serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
```



TCP三次握手建立连接

1）客户TCP发送SYN 报文段（SYN=1, ACK=0）
- 给出客户选择的起始序号 x
- 不包含数据

2）服务器TCP发送SYN, ACK报文段（SYN=ACK=1）（服务器端分配缓存和变量）
- 给出服务器选择的起始序号 y
- 确认客户的起始序号(x+1)
- 不包含数据

3）客户发送ACK报文段（SYN=0，ACK=1）（客户端分配缓存和变量）
- 确认服务器的起始序号(y+1)
- 可能包含数据（捎带）

如何选择起始序号

➢为什么起始序号不从0开始?
- 若在不同的时间、在同一对套接字之间建立了连接，则新、旧连接上的序号有重叠，旧连接上重传的报文段会被误以为是新连接上的报文段

➢可以随机选取起始序号吗?
- 若在不同时间、在同一对套接字之间建立了连接，且新、旧连接上选择的起始序号x和y相差不大，则新、旧连接上传输的序号仍然可能重叠

➢结论：必须避免新、旧连接上的序号产生重叠

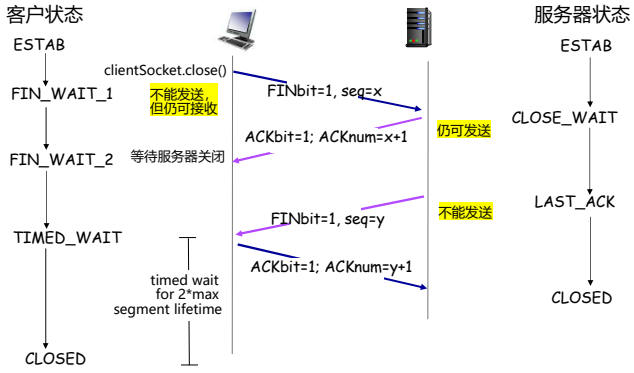
TCP起始序号的选择

➢基于时钟的起始序号选取算法:
- 每个主机使用一个时钟，以二进制计数器的形式工作，每隔ΔT时间计数器加1
- 新建一个连接时，以本地计数器值的最低32位作为起始序号
- 该方法确保连接的起始序号随时间单调增长

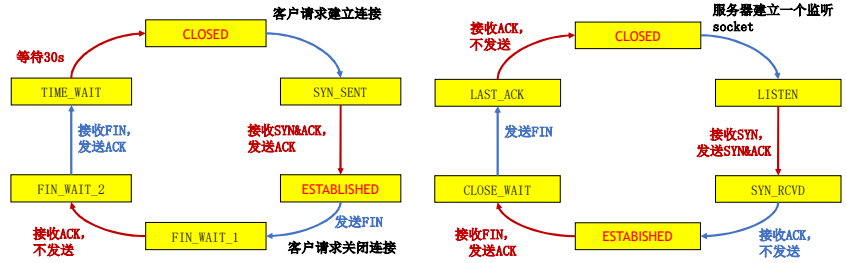
➢ΔT取较小的值（4微秒）：
- 确保发送序号的增长速度，不会超过起始序号的增长速度

➢使用较长的字节序号（32位）：
- 确保序号回绕的时间远大于分组在网络中的最长寿命

关闭TCP连接



C/S经历的TCP状态序列



客户生命周期（状态图）

服务器生命周期（状态图）

SYN洪泛攻击

➢TCP 实现的问题：

- 服务器在收到SYN段后，发送SYNACK段，分配资源
- 若未收到客户ACK段，服务器超时后重发SYNACK段
- 服务器等待一段时间（称SYN超时）后丢弃未完成的连接，SYN超时的典型值为30秒~120秒

➢ SYN 洪泛攻击：

- 攻击者采用伪造的源IP地址，向服务器发送大量的SYN段，却不发送ACK段
- 服务器为维护一个巨大的半连接表耗尽资源，导致无法处理正常客户的连接请求，表现为服务器停止服务

TCP端口扫描

➢TCP端口扫描的原理：

- 扫描程序依次与目标机的各个端口建立TCP连接
- 根据获得的响应来收集目标机器信息

➢在典型的TCP端口扫描过程中，发送端向目标端口发送SYN报文段：

- 若收到SYN+ACK段，表明目标端口有服务运行
- 若收到RST段，表明目标端口上没有服务在运行
- 若什么也没收到，表明路径上有防火墙，有些防火墙会丢弃来自外网的SYN报文段

➢FIN扫描试图绕过防火墙，发送端向目标端口发送FIN报文段：

- 若收到ACK=1、RST=1的TCP段，表明目标端口上没有服务在监听
- 若没有响应，表明有服务在监听（RFC 973的规定）
- 有些系统的实现不符合RFC 973规定，如在Microsoft的TCP实现中，总是返回ACK=1、RST=1的TCP段。

本章内容

6.1 传输层服务
6.2 套接Socket字编程
6.3 传输层复用和分用
6.4 无连接传输：UDP
6.5 面向连接的传输：TCP
6.6 TCP拥塞控制
6.7 拥塞控制的发展
6.8 传输层协议的发展

1. 理解拥塞控制
2. 拥塞控制要解决的问题
3. 拥塞控制调节策略
4. TCP的公平性

1. 理解网络拥塞---类比: 交通拥堵

交通拥堵:

- 起因：大量汽车短时间内进入路网，超出路网的承载能力
- 表现：道路通行能力下降，车速变慢，甚至完全停滞
- 措施：减少车辆进入路网(交通管制)

网络拥塞:

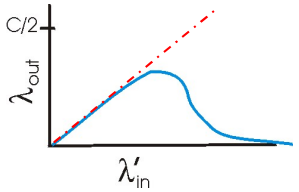
- 起因：大量分组短时间内进入网络，超出网络的处理能力
- 表现：分组延迟增大，网络吞吐量下降，甚至降为0
- 措施：减少分组进入网络(拥塞控制)

流量控制与拥塞控制的异同:

- 流量控制：限制发送速度，使不超过接收端的处理能力
- 拥塞控制：限制发送速度，使不超过网络的处理能力

1. 理解网络拥塞---拥塞后果

- 网络拥塞会造成:
  - 丢包：由路由器缓存溢出造成
  - 分组延迟增大：链路接近满载造成
- 大量网络资源用于:
  - 重传丢失的分组
  - （不必要地）重传延迟过大的分组
  - 转发最终被丢弃的分组
- 结果:
  - 进入网络的负载很重，网络吞吐量却很低



1. 理解网络---拥塞控制的常用方法

- 网络辅助的拥塞控制
  - 路由器向端系统提供显式的反馈，例如:
    - 设置拥塞指示比特
    - 给出发送速率指示

  ATM、X.25采用此类方法
- 端到端拥塞控制
  - 传输层不向端系统提供反馈
  - 端系统通过观察丢包和延迟，自行推断拥塞的发生
  - TCP采用此类方法

2. TCP拥塞控制要解决的问题

- TCP使用端到端拥塞控制机制:
  - 发送方根据自己感知的网络拥塞程度，限制其发送速率
- 需要回答三个问题:
  - 发送方如何感知网络拥塞?
  - 发送方采用什么机制来限制发送速率?
  - 发送方感知到网络拥塞后，采取什么策略调节发送速率?

拥塞检测和速率限制

发送方如何感知拥塞?

➢发送方利用丢包事件感知拥塞:
- 拥塞造成丢包和分组延迟增大
- 无论是丢包还是分组延迟过大，对于发送端来说都是丢包了

➢丢包事件包括:
- 重传定时器超时
- 发送端收到3个重复的ACK

发送方采用什么机制限制发送速率?

➢发送方使用拥塞窗口cwnd限制已发送未确认的数据量:

LastByteSent-LastByteAcked ≤ cwnd

$$rate = \frac{cwnd}{RTT} \quad Bytes/sec$$

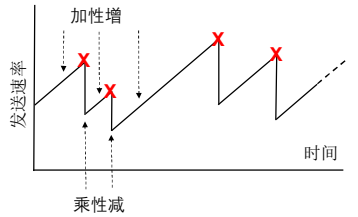
➢cwnd随发送方感知的网络拥塞程度而变化

3. 拥塞窗口的调节策略（发送方）：

AI and MD 策略

➢加性增（Additive Increase）
- 若无丢包，每经过一个RTT，将cwnd增大一个MSS，直到检测到丢包
- 目的：缓慢增大发送速率，避免振荡

➢乘性减（Multiplicative Decrease）
- 发送方检测到丢包后，将cwnd的大小减半（但不能小于一个MSS）
- 目的：迅速减小发送速率，缓解拥塞



cwnd呈锯齿状变化

AIMD（Additive Increase Multiplicative Decrease）

3. 拥塞窗口的调节策略（发送方）：

1）TCP慢启动/慢开始:

在新建的连接(或沉寂了一段时间的连接)上，以什么速率发送数据(此时接收窗口达最大值)？

➢早期的TCP协议:
- 发送端仅以接收窗口大小限制发送速率，网络经常因为拥塞而崩溃!

➢采用“加性增AI”增大发送窗口，太慢!
- 在新建连接上，令cwnd = 1 MSS，起始速度= MSS/RTT
- 然而，网络中的可用带宽可能远大于MSS/RTT

➢“慢启动”的基本思想:
- 在新建连接上指数增大cwnd，直至检测到丢包（此时终止慢启动）
- 希望迅速增大cwnd至可用的发送速度

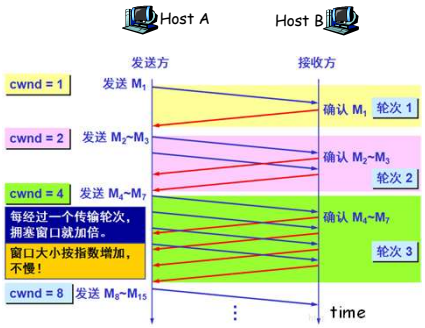
慢启动/开始的实施

➢慢启动的策略:
- 每经过一RTT，将cwnd加倍

✓慢启动的具体实施:
- 每收到一个ACK段(重传的不算)，cwnd增加一个MSS
- 只要发送窗口允许，发送端可立即发送下一个报文段

✓特点：低速开始，按指数增大发送速率

✓慢启动比谁“慢”？
- 与早期TCP按接收窗口发送数据的策略相比，采用慢启动后发送速率的增长较慢。



慢启动（Slow_start）

TCP拥塞控制的策略

- 发送方维护变量 ssthresh 门限
- 发生丢包或收到3个重复ACK时，ssthresh=cwnd/2
- ssthresh是从慢启动转为拥塞避免的分水岭：
  - cwnd低于该门限时，执行慢启动
  - cwnd高于该门限：执行拥塞避免

2）拥塞避免阶段：

每当收到ACK，拥塞窗口线性增长：

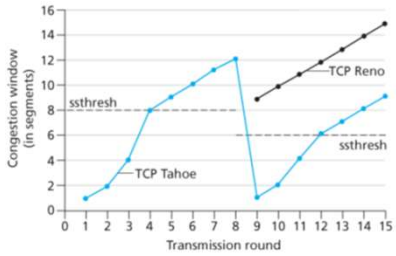
- cwnd=cwnd+MSS*(MSS/cwnd)

3）快速恢复：

- 检测到丢包后，ssthresh=cwnd/2

TCP Tahoe实现：cwnd=1 MSS，慢启动

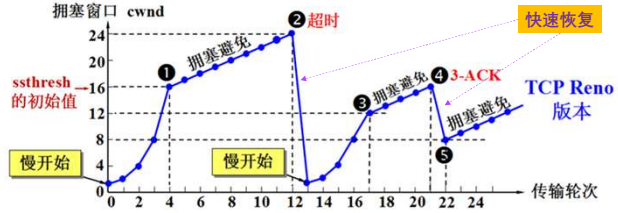
Slow start Congestion Avoidance fast recovery



- 检测到3个重复的ACK后：ssthresh=cwnd/2
- TCP Reno实现：

cwnd=ssthresh+3，线性增长

TCP拥塞控制示例



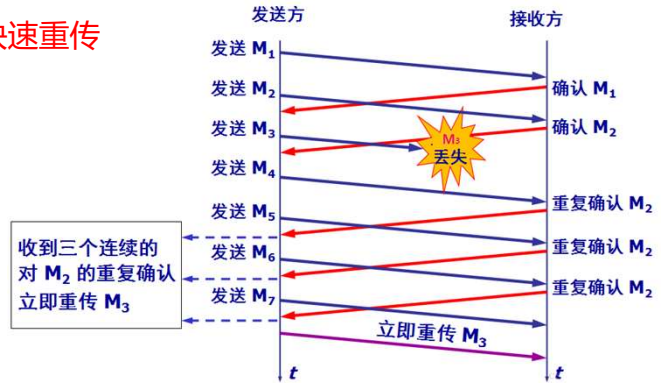
当 TCP 连接进行初始化时，将拥塞窗口置为 1。图中的窗口单位不使用字节而使用报文段。

假设：慢开始门限的初始值设置为 16 个报文段，即 ssthresh = 16。

https://blog.csdn.net/qq_32998153/article/details/79686802

TCP拥塞控制示例

4）快速重传



拥塞控制区分不同的丢包事件

超时和收到3个重复的ACK，它们反映出来的网络拥塞程度是一样的吗？ 当然不一样！

✓超时：说明网络交付能力很差

✓收到3个重复的ACK：说明网络仍有一定的交付能力

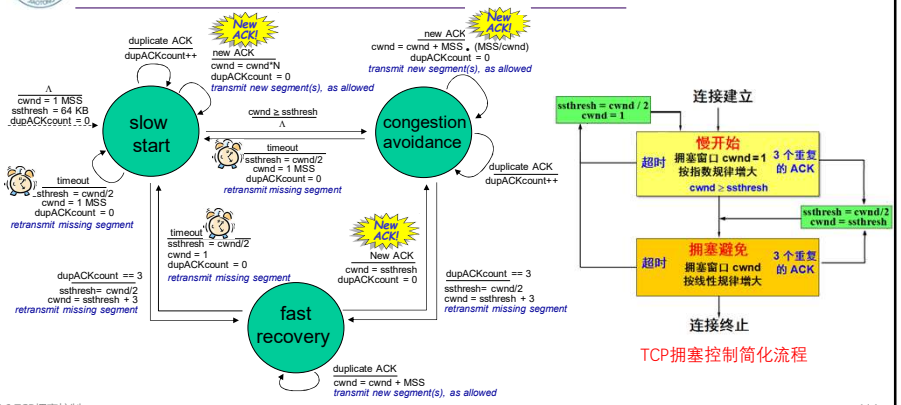
目前的TCP实现区分上述两种不同的丢包事件：

- 收到3个重复的ACK：
  - MD设置门限ssthresh = cwnd/2
  - 将cwnd降至ssthresh+3
  - 使用AI调节cwnd
- 超时：
  - MD设置门限ssthresh = cwnd/2
  - cwnd=1MSS
  - 使用慢启动增大cwnd至门限
  - 使用AI调节cwnd

TCP发送端的事件与动作

State	Event	TCP Sender Action	Commentary
慢启动 (SS)	收到新的确认	cwnd = cwnd *N, If (cwnd > ssthresh) set state to "Congestion Avoidance"	每经过一个RTT，cwnd加倍
拥塞避免 (CA)	收到新的确认	cwnd = cwnd+MSS * (MSS/cwnd)	每经过一个RTT，cwnd增加一个MSS
SS or CA	收到3个重复的确认	ssthresh = cwnd/2, cwnd = Threshold+3, Set state to "Congestion Avoidance"	cwnd减半，然后线性增长（AI）
SS or CA	超时	ssthresh = cwnd/2, cwnd = 1 MSS, Set state to "Slow Start"	cwnd降为一个MSS，进入慢启动
SS or CA	收到一个重复的确认	统计收到的重复确认数	cwnd 和 ssthresh 都不变

TCP拥塞控制状态机



TCP拥塞控制简化流程

TCP连接的吞吐量

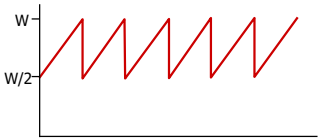
一个长期存活的TCP连接，平均吞吐量是多少?

- 忽略慢启动阶段（该阶段时间很短），只考虑拥塞避免阶段和快速恢复
- 令W=发生丢包时的拥塞窗口，此时有：

  throughput = W/RTT
- 发生丢包后调整 cwnd=W/2，此时有：

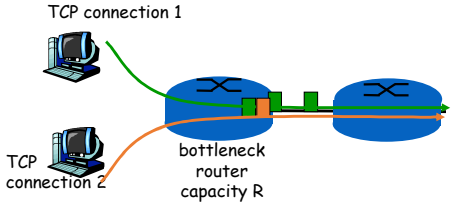
  throughput=W/2RTT
- 假设在TCP连接的生命期内，RTT 和 W几乎不变，有：

  Average throughout=0.75 W/RTT=(3/4)(W/RTT)



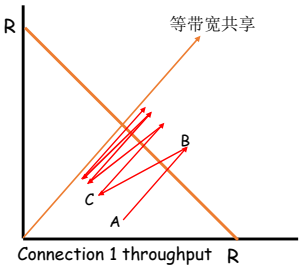
4. TCP的公平性

- 公平性目标:
  - 如果K条TCP连接共享某条带宽为R的瓶颈链路，每条连接应具有平均速度R/K。



为什么TCP是公平的

- 考虑两条竞争的连接（各种参数相同）共享带宽为R的链路:
  - 加性增AI、：连接1和连接2按照相同的速率增大各自的拥塞窗口，得到斜率为1的直线
  - 乘性减MD：连接1和连接2将各自的拥塞窗口减半



TCP公平性更复杂的情形

- 若相互竞争的TCP连接具有不同的参数（RTT、MSS等），不能保证公平性
- 若应用（如web）可以建立多条并行TCP连接，不能保证带宽在应用之间公平分配，比如：
  - 一条速率为R的链路上有9条连接
  - 若新应用建立一条TCP连接，获得速率 R/10
  - 若新应用建立11条TCP，可以获得速率约 R/2

关于TCP和UDP的思考

1) TCP提供可靠传输、流量控制、拥塞控制，而UDP都没有，能否说TCP服务优于UDP服务?
2) 多媒体应用希望的传输层服务是：带宽有保证、延迟有保证、顺序有保证、但能忍受一些丢包，TCP或UDP能够满足多媒体应用的需求吗?
3) 现实中的多媒体应用，有的使用TCP、有的使用UDP（分别是哪些？），它们分别出于什么考虑选择采用TCP或UDP?
4) 如何阻止UDP流量压制TCP流量?

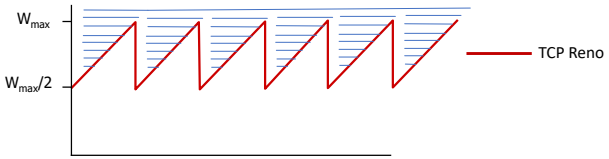
本章内容

6.1 传输层服务
6.2 套接字Socket编程
6.3 传输层复用和分用
6.4 无连接传输：UDP
6.5 面向连接的传输：TCP
6.6 TCP拥塞控制
6.7 拥塞控制的发展
6.8 传输层协议的发展

1. TCP CUBIC
2. Google BBR
3. Data Center TCP
4. DCCP

经典TCP拥塞控制的性能问题

- 核心问题：在探测满载窗口的过程中，如何增加拥塞窗口以尽可能利用网络带宽?
- TCP Reno线性增大拥塞窗口，探测当前可用网络带宽，即每经过一个RTT，拥塞窗口增加一个MSS；当端到端时延带宽乘积（BDP）较大时，拥塞窗口增长过慢，导致信道无法满载

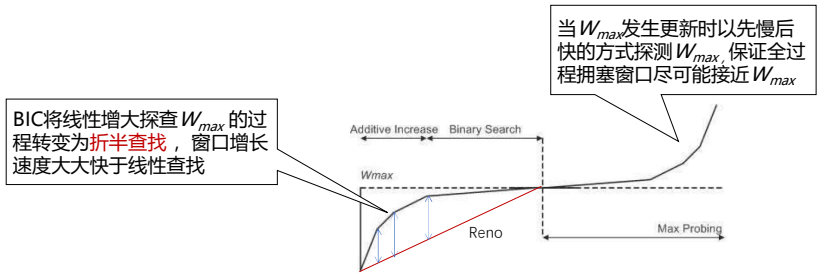


阴影部分表示信道带宽未被充分利用

TCP-BIC: Binary Increase Congestion

- BIC算法对满载窗口进行二分查找：
  - 如发生丢包时窗口大小是W_1，为保持满载而不丢包，满载窗口应小于W_1
  - 如检测到丢包并将窗口乘性减小为W_2，则满载窗口应大于W_2
- 窗口更新受ACK时钟驱动，即以RTT为更新间隔时间
- 二分查找:
  - 在ACK时钟的驱动下，将拥塞窗口置为(W_1 + W_2)/2(新的W_2值)，不断逼近满载窗口
- 最大探查:
  - 如窗口再次达到W_1而没有丢包，说明满载窗口大于W_1，则以逼近W_1的镜像过程增大拥塞窗口

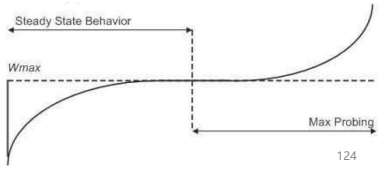
TCP-BIC 图解



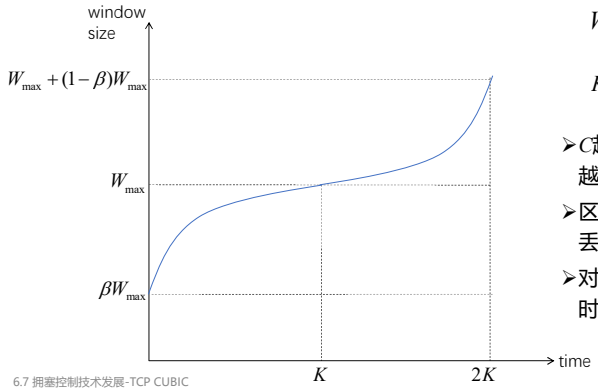
- BIC存在带宽不公平性问题
  - BIC以ACK时钟驱动拥塞窗口的更新，RTT较短的连接会更快到达满载窗口，占据更多的带宽，产生不公平性问题（RTT-fairness）

TCP CUBIC

- CUBIC将BIC算法连续化，用三次函数拟合BIC算法曲线
- 拥塞窗口成为距上次丢包的时间t的函数，t取值位于两次丢包之间，不再根据RTT间隔来确定调整窗口的时机，避免了RTT不公平问题
- 三次函数增长分为两个阶段：
  - Steady State Behavior阶段：以凹函数增长逼近最近一次丢包时窗口；
  - Max probing阶段：以凸函数增长探测当前满载窗口
- 拥塞窗口在绝大多数时间内接近W_{max}，保持了较高的发送效率
- CUBIC已实现在Linux 2.6.18中



CUBIC算法曲线



$$W(t)=C(t-K)^3+W_{max}$$

$$K=\sqrt[3]{\frac{W_{min}}{C}},\quad W_{min}=\beta W_{max}$$

- C越大，探测到最大窗口的时间越短，决定了对称区间的高度
- 区间起始时刻为最近一次检测到丢包的时间
- 对称区间的宽度不再依赖于ACK时钟

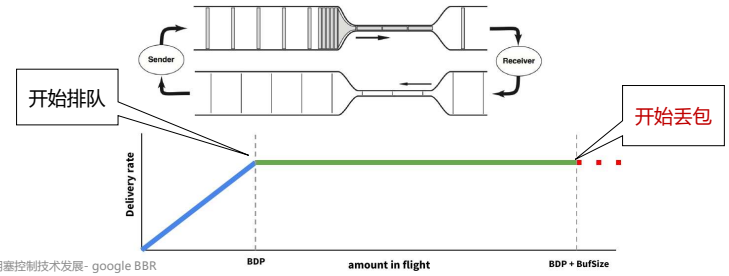
本章内容

6.1 传输层服务
6.2 套接字Socket编程
6.3 传输层复用和分用
6.4 无连接传输：UDP
6.5 面向连接的传输：TCP
6.6 TCP拥塞控制
6.7 拥塞控制的发展
6.8 传输层协议的发展

1. TCP CUBIC
2. Google BBR
3. Data Center TCP
4. DCCP

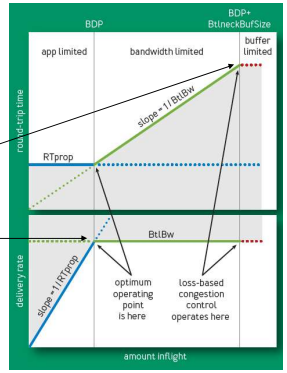
拥塞与瓶颈链路带宽

- 拥塞与瓶颈链路带宽：
  - 瓶颈链路带宽BtlBw，决定了端到端路径上的最大数据投递速率
  - 拥塞窗口大于BtlBw时，瓶颈链路处会形成排队，导致RTT延长(直至超时)



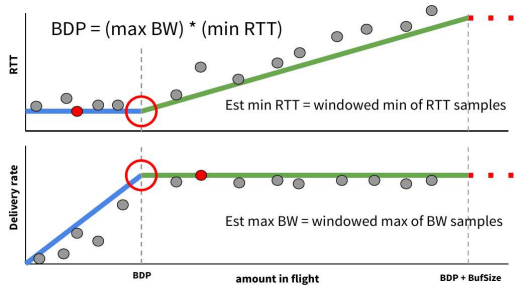
BBR: Bottleneck Bandwidth and Round-trip propagation time

- 瓶颈链路带宽BtlBw：不会引起路由器缓存排队的最大发送速率
- RTprop：往返时间
- BDP=BtlBw RTprop
- 经典拥塞控制状态机以丢包事件为驱动，探测阶段将瓶颈链路上的缓冲区填满直至丢包，并以此为依据判断是否进行被动的乘性减小
- BBR试图测量图中左侧优化点的BtlBw，尽量将cwnd收敛到实际BtlBw，从而避免出现丢包，属于主动探测
- 以图中BDP竖线为分界点，右侧测得Rtprop会因瓶颈链路发生排队而逐渐增长



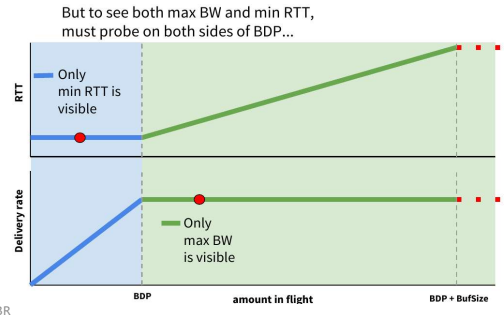
优化点的近似观测

用过去10秒内的最小RTT（min RTT）和最大投递率（max BW），分别近似RTprop和BtlBw，并依据这两个值估算当前BDP



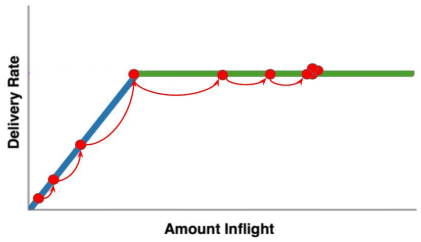
Max BW和min RTT不能同时被测得

- 要测量最大带宽，就要把瓶颈链路填满，此时buffer中存在排队分组，延迟较高
- 要测量最低延迟，就要保证链路队列为空，网络中分组越少越好，cwnd较小



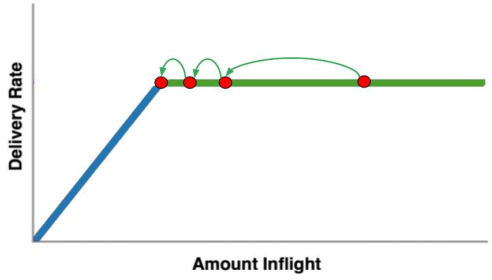
BDP检测：启动阶段（START_UP）

- 当连接建立时，类似TCP的慢启动，指数增加发送速率，尽可能快地占满管道
- 若经过三次发现投递率不再增长，说明已达到BtlBw，瓶颈链路处分组已开始排队（事实上此时占的是三倍BDP）



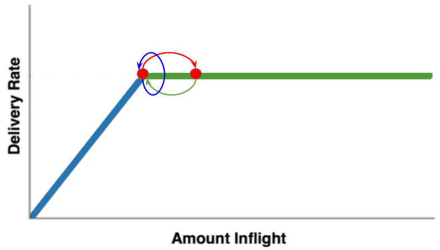
BDP检测：排空阶段（DRAIN）

指数降低发送速率（相当于是startup的逆过程），将多占的两倍buffer慢慢排空



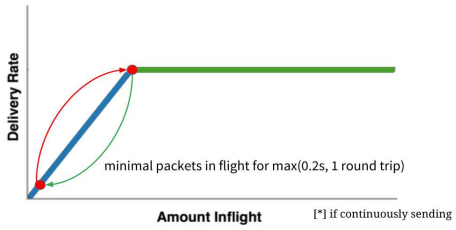
BDP检测：瓶颈带宽探测(PROBE_BW)

- 进入稳定状态后，先在一个RTT内增加发送速率，探测最大带宽
- 如果RTT没有变化，再减小发送速率，排空前一个RTT多发出来的包
- 后面6个周期使用更新后的估计带宽发送

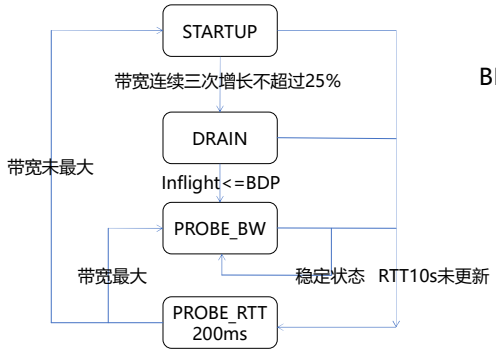


BDP检测：时延探测(PROBE_RTT)

- 每过10秒，如果估计的RTprop不变，就进入RTprop探测阶段
- 在这段占全过程2%的时间内，cwnd固定为4个包
- 测得的RTprop作为基准，用以判断带宽检测阶段瓶颈链路中是否发生排队
- 为抵消此阶段牺牲的发送速率，结合pacing_gain数组在后期短时间增加发送速率，保证对BtlBw的及时观测
- pacing_gain数组用来微调每个状态下不同时段的拥塞窗口大小



BBR状态机



BBR已实现在Linux 4.9中

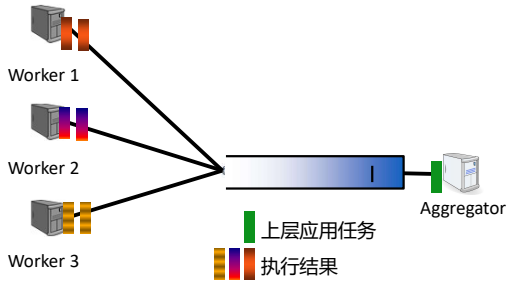
本章内容

6.1 传输层服务
6.2 套接字Socket编程
6.3 传输层复用和分用
6.4 无连接传输：UDP
6.5 面向连接的传输：TCP
6.6 TCP拥塞控制
6.7 拥塞控制的发展
6.8 传输层协议的发展

1. TCP CUBIC
2. Google BBR
3. Data Center TCP
4. DCCP

数据中心 (DC) 的通信模式

- 数据中心内部有一类经典的通信模式-Partition/Aggregate
  - Aggregator将上层应用任务划分给下层的worker执行



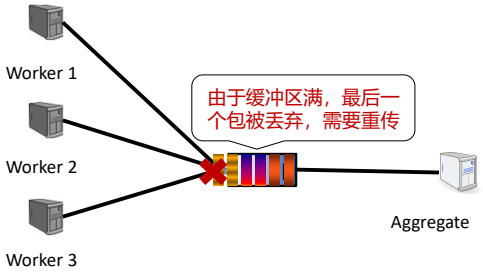
数据中心的流量特点

- 90%+的流量基于TCP连接，根据流量大小可以分为：
  - 时延敏感的短流（50KB-1MB）
  - 吞吐敏感的长流（1MB-50MB）
- 此外，还存在大量突发流量
  - 时间<1s
  - 大小<2KB

以上数据来源于:
[1] Alizadeh M, Greenberg A, Maltz D A, et al. Data center tcp (dctcp)[C]//Proceedings of the ACM SIGCOMM 2010 Conference. 2010: 63-74.

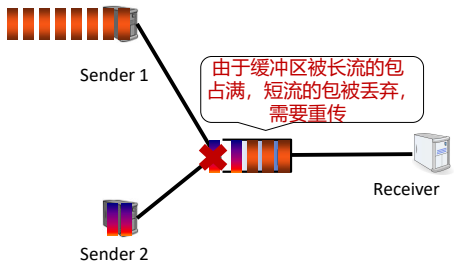
数据中心的性能问题

1) Incast
   - 在一个很短的时间内，大量流量同时到达交换机的一个端口，导致缓冲区被占满，最终导致丢包
   - 在并发流量很大的情况下，即使每条流的包很小，也会产生Incast问题



数据中心的性能问题

2) Queue Buildup
   - 长流和短流同时通过交换机的同一个端口时，由于长流占用较多的缓冲区空间，导致短流延迟增大，甚至丢包
3) Buffer Pressure
   - 交换机的不同端口通常共享同一块缓冲区，即使长流和短流通过不同的端口，短流通过的端口也会出现缓冲区不足的问题



数据中心需要什么样的传输层协议

➢容忍高突发流量
- 在Partition/Aggregate通信模式中，所有Worker几乎会在同一时间向Aggregator返回执行结果，产生很高的突发流量

➢低时延
- 数据中心有大量时延敏感的短流，如网页搜索等

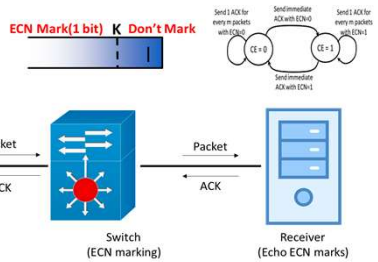
➢高吞吐
- 数据中心有大量吞吐敏感的长流，如文件传输、分布式机器学习中神经网络模型参数的传输等

DCTCP核心思想

➢根据网络拥塞程度精细地减小发送窗口：
- 一旦发现拥塞，发送窗口减至原窗口的（1-α/2），α 反映了拥塞程度，而TCP中 α总为1

➢根据交换机队列的瞬时长度标记ECN（explicit congestion notification）
- 使用显式的拥塞反馈能够更好地控制突发流量

$$cwnd = \begin{cases} (1-\frac{\alpha}{2}) \times cwnd, & \text{when congestion happens} \\ cwnd + 1, & \text{when there is no congestion} \end{cases}$$

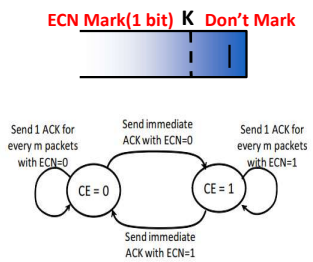


DCTCP协议要点

➢交换机
- 当队列长度超过K时，给随后到来的包标记ECN

➢接收端
- 仅当被标记的报文出现或消失时才立即发送ACK，否则采取Delay ACK的策略

➢发送端
- 每个RTT更新一次发送窗口
- $\alpha \leftarrow (1-g)\alpha + \alpha F$，这个值反应了拥塞程度
- 其中$F = \frac{\#\ of\ marked\ ACKs}{Total\ \#\ of\ ACKs}$
- $Cwnd \leftarrow \left(1-\frac{\alpha}{2}\right) * Cwnd$

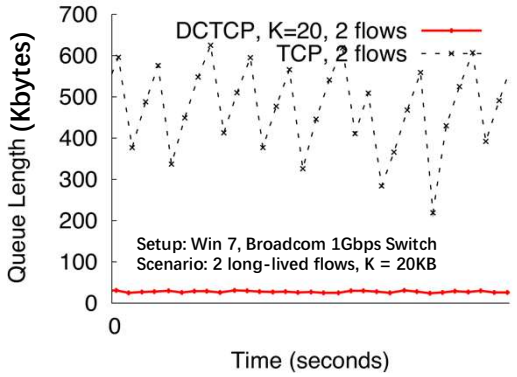


➢DCTCP的实现：
- 只需30行左右的代码改动
- 在交换机上设置参数K

DCTCP和TCP的比较

➢交换机中的队列长度：
- DCTCP能将队列长度稳定地维持在一个很低的水平
- 而TCP的队列长度不仅高，而且波动很大

➢因此，DCTCP更适用于现代数据中心



为什么DCTCP能够奏效?

- 容忍高突发流量
  - DCTCP维持了较低的队列长度，可以留出较大的缓冲区给突发流量
  - 采用激进的标记策略，使得发送端在丢包之前就感知到拥塞
- 低时延
  - 由于队列长度较短，也减少了包在队列中的排队时延
- 高吞吐
  - DCTCP根据拥塞程度精确调节窗口，使得发送窗口的变化比较平滑，不会出现吞吐量骤降的情形

本章内容

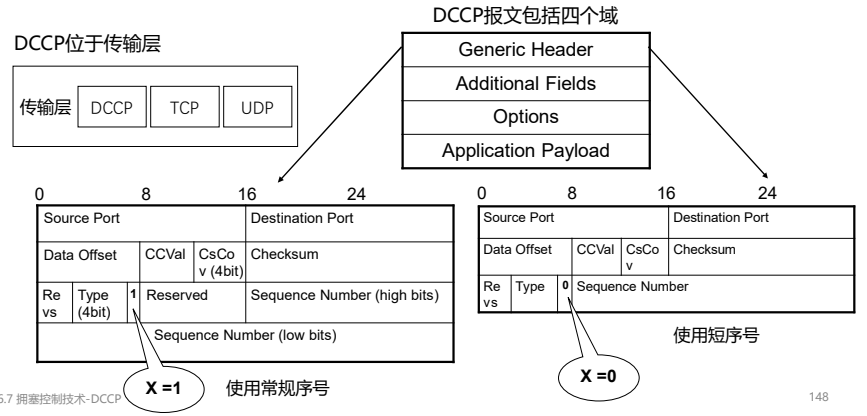
6.1 传输层服务
6.2 套接字Socket编程
6.3 传输层复用和分用
6.4 无连接传输：UDP
6.5 面向连接的传输：TCP
6.6 TCP拥塞控制
6.7 拥塞控制的发展
6.8 传输层协议的发展

1. TCP CUBIC
2. Google BBR
3. Data Center TCP
4. DCCP

UDP的问题和改进方案

- 一些容忍少量丢包、但延迟敏感的应用，适合采用UDP传输：
  - 如视频会议、在线游戏等
- 然而，UDP没有拥塞控制机制：
  - 拥塞发生时，UDP既感知不到拥塞，也不会降低其发送速率，从而影响应用性能，也影响网络的整体性能
  - 对TCP协议极不友好，损害网络公平性（TCP让出的带宽都被UDP占了）
- 解决方案：
  - 给UDP增加拥塞控制机制
- DCCP=UDP+拥塞控制
  - 增加ACK报文、包序列号、面向连接等拥塞控制手段
- DCCP提供不可靠数据报流服务：
  - UDP报文段可能丢失、乱序到达
  - 无流量控制
- DCCP提供模块化的拥塞控制：
  - 应用可以根据需要，选择不同的拥塞控制机制

DCCP的报文格式

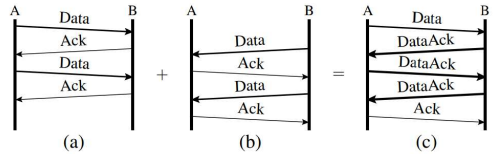


DCCP的报文类型

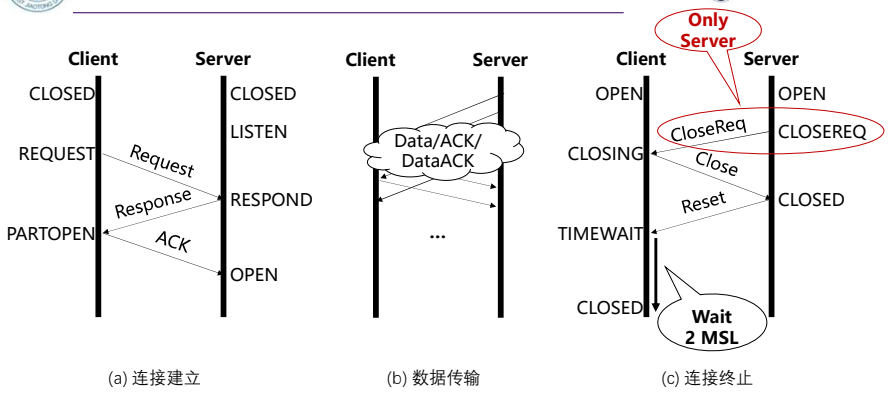
连接初始化	DCCP-Request	An open connection request(from client)
	DCCP-Response	An open connection response(from server)
数据传输	DCCP-Ack	Transmit ack info
	DCCP-Data	Transmit data
	DCCP-DataAck	DCCP-Data + DCCP-Ack
连接终止	DCCP-CloseReq	An close connection request(from server)
	DCCP-Close	An close connection request(from client)
	DCCP-Reset	Destroy connection
重新同步	DCCP-Sync, DCCP-SyncAck	Recover synchronization after bursts of loss and recover from half-open connections

DCCP连接

- DCCP是面向连接的，两个主机之间有一个全双工连接：
  - 由两个方向的半连接组成
  - 以A to B方向的半连接为例
    - A向B发送Data报文段
    - B向A发送ACK报文段
  - 同一方向的ACK段和Data段可以合并成DataAck报文段



DCCP的交互流程



DCCP的拥塞控制机制

- DCCP连接支持拥塞控制，应用可以根据需要选择不同的拥塞控制机制：
  - 组成一个连接的两个半连接，可以选择不同的拥塞控制机制
- 不同的拥塞控制机制由一个1字节的CCID来标识，连接建立时两个终端协商确定
- 目前DCCP的RFC文档定义了两个CCID，分别是2和3，其余的值保留
  - CCID2：TCP-like Congestion Control
  - CCID3：TCP-Friendly Rate Control(TFRC)

DCCP的拥塞控制机制

- TCP-like Congestion Control（CCID2）
  - 发送方维持一个发送窗口，并且不停地发包，直到没有可用的发送窗口
  - 接收方对收到的包回复ACK报文，包括一段时间内收到的所有包的序号（用于支持SACK机制）
  - 丢包或者ECN标记表示出现拥塞，此时发送端需要将发送窗口减半
- TCP-Friendly Rate Control（CCID3）
  - 接收方计算丢包率，报告给发送方
  - 发送方根据丢包率计算发送速率，并根据发送速率发包
- 相比而言，CCID3可以更平滑地调节发包速率

本章内容

6.1 传输层服务
6.2 套接字Socket编程
6.3 传输层复用和分用
6.4 无连接传输：UDP
6.5 面向连接的传输：TCP
6.6 TCP拥塞控制
6.7 拥塞控制的发展
6.8 传输层协议的发展
1. SCTP
2. MPTCP
3. QUIC

SCTP: Stream Control Transmission Protocol

SCTP是一个IP协议（协议号132）之上的、可靠的、面向控制信令的、传输层协议。SCTP可在IP网络之上为电信级信令传输提供高效、可靠的信令传输服务，例如5GC中的N4接口就使用了SCTP协议。

SCTP的诞生是为了弥补TCP/UDP在电信网络控制信令传输场景中的不足，它们都难以完全满足在电信网络中信令承载的要求。包括：

- UDP是一种无连接的不可靠传输协议，自然无法满足信令对传输质量的要求。
- TCP虽然是一种基于连接的可靠传输协议，但同时也具有队头阻塞、实时性差、支持多归属困难、易受DDoS攻击的缺陷。

为了解决TCP/UDP协议在传输实时信令时所面临的不可靠传输和时延等问题，SCTP 协议设计了以下的核心特性，包括：

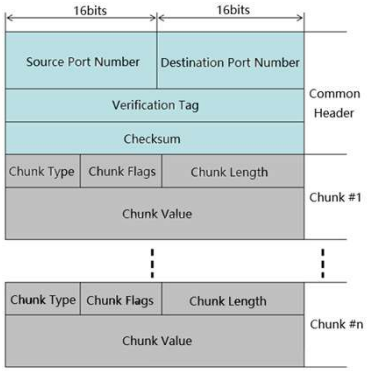
① 适当的拥塞控制；② 防止泛滥和伪装攻击；③ 更优的实时性能；④ 多属主（Multi-homing）特性支持；⑤ 多流（Multi-streaming）特性支持等等。

SCTP: Stream Control Transmission Protocol

- **SCTP:**

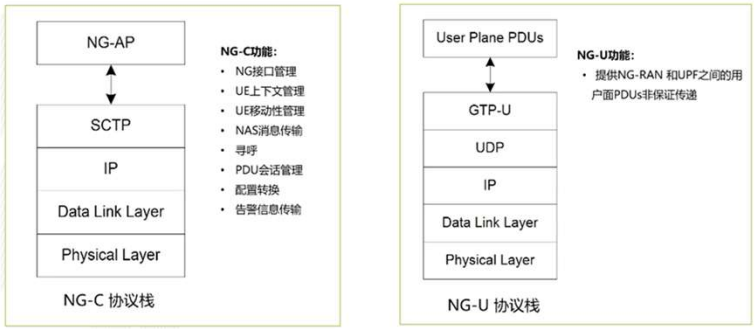
和TCP类似，SCTP是面向连接、端到端、全双工、带有流量和拥塞控制的可靠传输协议。SCTP连接称为关联。SCTP关联通过4次握手建立，3次挥手。

相对于TCP的3次握手建立连接，SCTP的关联能够抵御拒绝服务(DoS)攻击，从而提高了安全性。数据只有在关联建立之后与关联关闭之前才可发送。SCTP的关联通过3次握手关闭，不支持类似TCP的半关闭连接。也就是在任何一方关闭关联后，对方即不再发送数据。



SCTP

NG接口协议栈



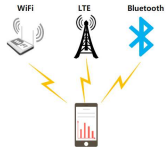
本章内容

6.1 传输层服务
6.2 套接字Socket编程
6.3 传输层复用和分用
6.4 无连接传输：UDP
6.5 面向连接的传输：TCP
6.6 TCP拥塞控制
6.7 拥塞控制的发展
6.8 传输层协议的发展

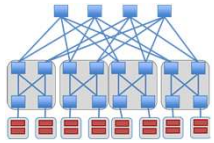
1. SCTP
2. MPTCP
3. QUIC

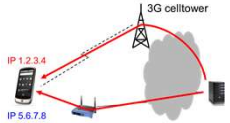
1. 传统TCP协议的不足与改进

- 传统TCP协议仅支持单路径传输，即只能利用终端主机上的一个网络接口传输数据
- 随着接入技术的发展，同时具备多个网络接口的网络设备已经越来越普及，多路径传输更适合当前的网络环境



移动网络


数据中心移动网络

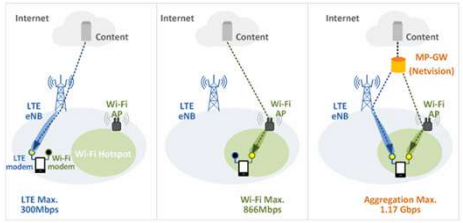


多路径传输示意图

- 移动网络和数据中心网络中的网络设备天然拥有多个网络接口，已具备实现多路径传输的物理基础
- 多路径TCP协议（MPTCP）应运而生，它可将单一数据流切分为若干子流，同时利用多条路径进行传输

MPTCP的优势

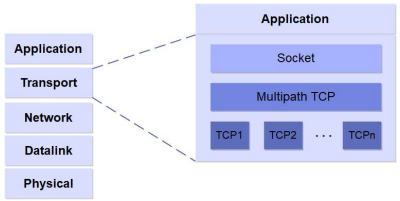
- 多径带宽聚合
  - 终端设备可以聚合不同路径上的可用带宽，以获得更高的网络带宽
- 提升传输的可靠性
  - 使用多条路径传输数据，可以有效避免因单条路径性能恶化或中断导致的应用连接中断
- 支持链路的平滑切换
  - 多路径传输方式允许终端在不同接入网络间快速、平滑地切换，选取当前链路质量最好的路径传输数据



图片来源：https://www.cnblogs.com/yhp-smarthome/p/7440855.html

MPTCP在网络体系结构的位置

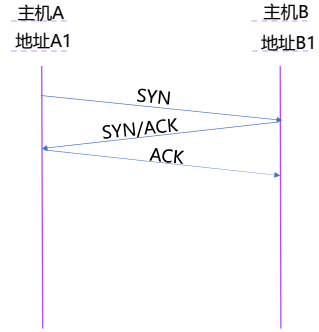
- MPTCP位于套接字和TCP之间：
  - 应用程序通过套接字调用MPTCP，MPTCP向应用程序提供单条连接的抽象，因而对应用层透明
  - MPTCP可在源主机和目的主机的多对网络接口间分别建立TCP连接，并将数据流分配到多条TCP连接上传输
  - MPTCP兼容并扩展了TCP协议：TCP基本头不变，只定义了新的选项，从而对网络层也是透明的



- MPTCP连接是一个或多个子流的集合：
  - 路径：本地主机与远程主机之间可用于建立连接的一个链路序列称为路径，使用四元组<本地IP地址，本地端口，远程IP地址，远程端口>表示
  - 子流：在单个路径上运行的 TCP 流称为子流，是MPTCP连接的组成部分

MPTCP连接管理

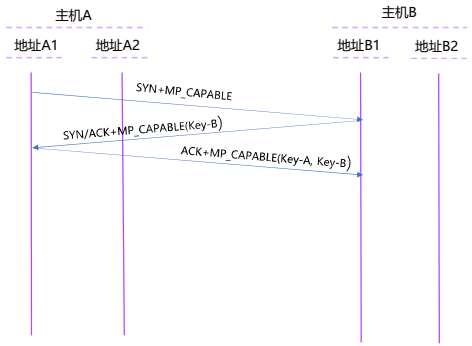
- 用于MPTCP连接管理的新字段：
  - MP_CAPABLE：建立MPTCP连接
  - MP_JOIN：附加新的子流到已有连接
  - ADD_ADDR：新增可用路径
  - REMOVE_ADDR：删除路径
  - MP_FASTCLOSE：关闭所有子流
- 如何建立MPTCP连接：
  - 首先初始化一条MPTCP连接（与建立常规TCP连接的过程相似）
  - 然后将MPTCP的其它子流附加到已经存在的MPTCP连接上



常规TCP连接的建立过程-三次握手

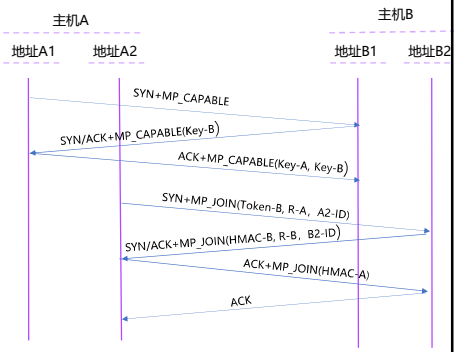
初始化MPTCP连接

- 初始化一条MPTCP连接（A1->B1）
  - 建立MPTCP连接的过程与建立常规TCP连接相似，不同之处在于启用了MP_CAPBLE字段，并交换了用于身份认证和建立子流的密钥信息
  - 主机A发出SYN报文段，其中启用了MP_CAPBLE字段，用以询问主机B是否支持MPTCP
  - 若主机B支持MPTCP，则响应SYN/ACK报文段，其中启用了MP_CAPBLE字段并附上自己的密钥
  - 主机A向主机B发送ACK报文段，并附上自己和主机B的密钥，至此MPTCP的连接初始化完成



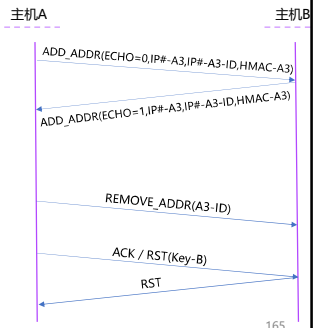
附加子流到MPTCP连接上

- 将新的子流（A2->B2）附加到已存在的MPTCP连接
  - 附加子流需要启用MP_JOIN字段
  - 主机A根据Key-B生成主机B的令牌（Token-B），该令牌指定当前子流需要附加到哪个MPTCP连接上，并将自身产生的随机数（R-A）和当前子流的地址编号（A2-ID）添加到SYN报文段中
  - 主机B根据前面MP_CAPBLE字段交换的密钥和随机数（R-A）产生认证信息（HMAC-B），与自身产生的随机数（R-B）和当前子流的地址编号（B2-ID)一同写入SYN/ACK报文段中
  - 主机A验证HMAC-B，无误后向主机B发送ACK报文段，并附上自己的认证信息（HMAC-A）
  - 主机B验证HMAC-A，无误后向主机A发送ACK报文段，至此子流建立完毕



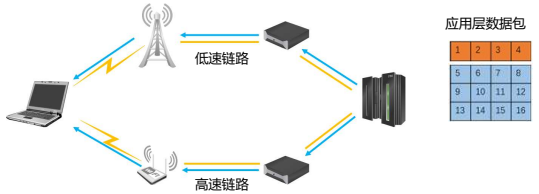
MPTCP路径管理和关闭连接

- 新增路径
  - 隐式新增：使用MP_JOIN方式新增子流
  - 显式新增：仅新增路径(并不启用新的子流)，使用ADD_ADDR字段
    - 主机A发送启用了ADD_ADDR字段的报文段，包含新的IP地址/端口对（IP#-A3)和对应的IP地址编号(IP#A3-ID)、认证信息（HMAC-A3），ECHO指示当前报文段是“发出”
    - 主机B验证HMAC-A3，无误后将路径信息记录下来，向主机A发送响应报文段，ECHO标注为“响应”，其它信息原样发回
- 删除路径
  - 主机A发送启用了REMOVE_ADDR字段的报文段，指定要删除的地址编号
- 关闭单个子流：与关闭常规TCP连接一致
- 关闭所有子流：
  - 主机A发送启用了MP_FASTCLOSE字段的报文段，其中附加了主机B的密钥用以身份认证
  - 主机B收到后，选择恰当时机关闭所有连接，并向主机A发出确认



MPTCP的数据调度

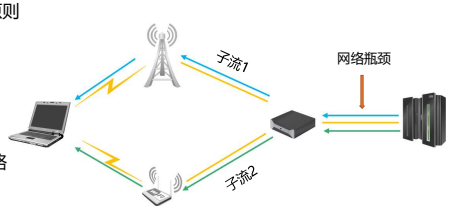
- 在多路径传输中，发送端将属于同一个数据流的数据包调度到不同的路径上传输，由于不同路径的差异，这些数据包往往无法按照发送顺序到达接收端



- 乱序到达的数据包需暂存在接收缓存中，直到接收缓中的数据包能够按序交付给上层应用，这既影响了数据传输的实时性，又影响了网络的吞吐量
- MPTCP根据拥塞窗口大小及路径延迟，将数据按比例分配给各个子流，尽力保证数据包按序到达接收端，降低数据乱序到达对网络性能产生的不利影响

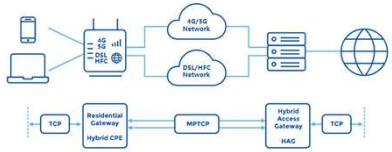
MPTCP的拥塞控制

- IETF MPTCP工作组提出的拥塞控制的设计目标
  - 提升吞吐量：多路径TCP连接的各个子流获得的总吞吐量，不应低于其最优路径上单路径TCP连接的吞吐量
  - 公平性：多路径TCP连接在同一网络瓶颈处的多个子流，不能过多地侵占其它单路径TCP连接的带宽
  - 均衡拥塞：多路径TCP连接应能在满足前两个准则的情况下，实现各个子流之间的负载均衡，尽可能将拥塞路径上的流量迁移到较好的路径上
- 公平性是关注的重点，MPTCP采用的是网络公平性原则
  - 举例：假设MPTCP的两个子流与一个常规TCP流共享一个网络瓶颈，则MPTCP两个子流获得的带宽应为瓶颈带宽的1/2，而不是2/3，这样才能保证MPTCP连接对常规TCP连接的友好性
- MPTCP实现网络公平性准则的方法：
  - 限制各个子流窗口增长速度的总和，不超过单路径TCP连接的窗口增长速度
  - 从而，MPTCP连接通过各子流获得的总吞吐量和单路径TCP连接获得的吞吐量相当



MPTCP的应用现状

- Linux内核已经支持MPTCP协议，当前最新版本为v0.95（ http://www.multipath-tcp.org/ ）
- iPhone和iPad上的Multipath TCP（https://support.apple.com/zh-cn/HT201373）
  - 在同时支持蜂窝移动数据网络和Wi-Fi网络的iOS设备中，Siri 尝试通过 Wi-Fi 建立 MPTCP 连接；如果连接成功，Siri 通过蜂窝移动数据建立备用连接；如果 Wi-Fi 不可用或不可靠，MPTCP 立即在后台切换到蜂窝移动数据网络
- 三星的手机设备 Galaxy S6 和 S6 Edge 已经在韩国KT运营网络中支持MPTCP协议
- Citrix公司的Web应用交付方案Netscaler 和F5公司的 BIG-IP 目前也同样支持使用MPTCP协议提升网络性能
- Tessares公司推出的商业化MPTCP解决方案可以在用户端和服务器端快速部署MPTCP服务



本章内容

6.1 传输层服务
6.2 套接字Socket编程
6.3 传输层复用和分用
6.4 无连接传输：UDP
6.5 面向连接的传输：TCP
6.6 TCP拥塞控制
6.7 拥塞控制的发展
6.8 传输层协议的发展

1. MPTCP
2. QUIC
3. SCTP

TCP存在的问题

- TCP 实现在操作系统内核中
  - 作为传输优化的最终受益者，应用无法对TCP进行修改
  - 操作系统的更新往往跟不上应用的需求和节奏
- TCP体系握手时延大
  - 互联网上的大趋势：低时延需求越来越强烈；加密流量占比越来越大
  - TLS(传输层安全性协议)+TCP的体系握手时延很大，传输前需要3个RTT进行握手

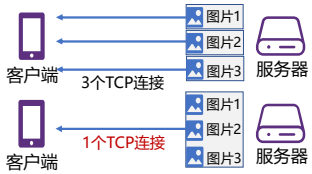


近年来加密流量占比迅速上升

TCP存在的问题

- TCP多流复用存在队头阻塞问题
  - 当前的应用普遍比较复杂，常需要同时传输多个元素
    - 例如：网页传输中，每个单独的图片即为一个数据流，不同数据流之间相互独立。为每个数据建立一个TCP连接很低效（尤其对于小流，单独建立连接成本高昂），因此出现了多流复用
  - TCP传输需要保持有序性
    - 出现丢包时，后面的数据需要等丢失的包重传完成才能使用，这就导致了队头阻塞

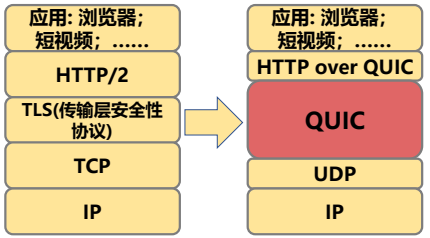

网页中的多个图片可以同时传输



上图：每个数据流使用单独的TCP连接
下图：多个数据流复用一个TCP连接

QUIC在网络体系结构的位置

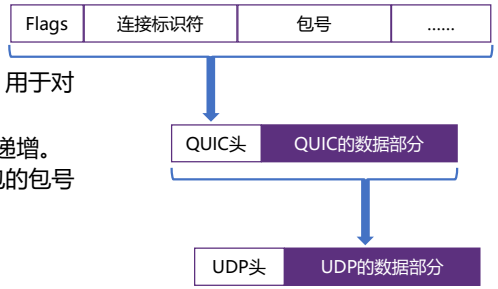
- 传统的传输架构
  - TCP提供数据传输服务
  - TLS（传输层安全性协议）对数据进行加密
  - HTTP协议定义如何发起请求-响应请求
  - 应用在HTTP之上实现
- 基于QUIC的传输架构
  - QUIC替代TCP、TLS和部分HTTP的功能
- QUIC实现在用户态中
  - 底层基于UDP实现
  - 拥塞控制是模块化的，可以方便地使用各种TCP拥塞控制算法，如Cubic等



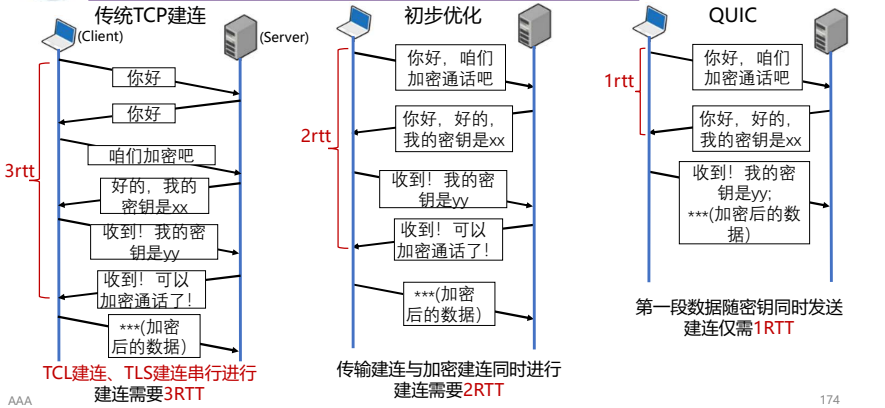
基于TCP的传输架构　　基于QUIC的传输架构

QUIC包格式介绍（简化）

- 部分相关字段：
  - 连接标识符(Connection ID)：用于对连接进行表示和识别
  - 包号(Packet Number)：单调递增。即同一个连接中，每个QUIC包的包号都不一样
- QUIC底层使用UDP进传输
  - QUIC包作为UDP的数据载荷
  - IANA（互联网数字分配机构）建议QUIC使用UDP的443端口



连接建立时延优化：简化演示

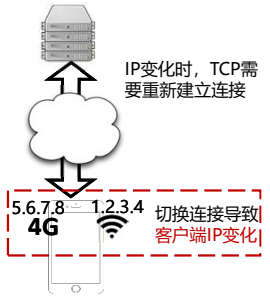


明确的包序号和更精确的RTT

- TCP重传歧义的问题：
  - TCP的重传包使用和原包相同的序号，因此可能某一序号被用了不止一次
  - TCP收到这一序号的ACK时，无法判断是针对哪个包的ACK，从而影响后续操作，如测量RTT的大小
- QUIC解决重传歧义的方法：
  - QUIC的packet number单调递增，对于重传包也会递增packet number
  - 每个packet number只会出现一次，ACK没有歧义！
  - QUIC接收端记录收到包与发出ACK之间的时延，并发馈给发送端，方便发送端更准确地测量RTT

IP/Port切换无需重新建立连接

- IP地址/端口发生变化时，TCP连接会断开
  - 例如手机WIFI断开时，常常自动转而使用移动信号
  - 此时，TCP会断连，需要应用进行处理
- QUIC支持IP/端口切换
  - QUIC使用Connection ID来表示每个连接
  - IP地址或端口的变化不影响对原有连接的识别
  - 客户IP地址或端口发生变化时，QUIC可以快速恢复
- 由传输层对连接的切换进行管理
  - 更符合互联网体系结构
  - 不再需要应用重复造轮子

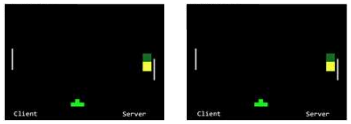


无队头阻塞的多流复用

➢多流复用时的队头阻塞问题
- TCP为保持数据的有序性，出现丢包时，会等待该数据到达后，再提交给上层应用
- 多流复用时，某个数据流的数据包丢失，会使得TCP连接上所有数据流都需要等待

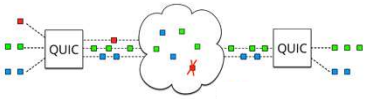
➢QUIC对队头阻塞问题的解决
- 在单个连接中，建立相互独立的多个QUIC流，某个流的数据包丢失不影响其它流的数据交付
- 分析：TCP保证了整个连接的数据交付的有序性（有序）；QUIC利用了各个流相互独立的特性，仅保持了流内部数据的有序性（部分有序），减少了不必要的等待

动画：同时发送两个流，黄颜色的流上有丢包



TCP丢包会阻塞所有后续的流

QUIC丢包只会影响相关的流



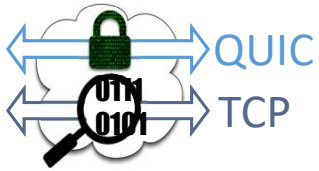
红色流的包丢失，不会阻塞绿色和蓝色的流

QUIC易于部署和更新

➢ 整个QUIC包被加密传输
- 保护用户数据隐私
- 避免被中间设备识别和修改

➢QUIC在用户态实现
- 与操作系统解耦，从而能和应用一同快速迭代

➢版本协商机制
- 由于QUIC的快速迭代特性，会同时存在众多QUIC版本
- 客户需要和服务器进行版本协商（不引入额外时延的协商机制）



中间设备无法识别和修改QUIC header中的信息

QUIC的发展状态

互联网发展史



- 截止2017年，互联网上7%的数据使用QUIC进行传送
- 2018年，IETF宣布，HTTP/3将弃用TCP协议，改为使用QUIC协议实现
- 2020年，华为在最新的HMS core网络加速套件中推出hQUIC
- 2020年，Facebook 宣布其超过75% 的网络流量使用 QUIC；……

正在推进的新特性（截止2020年底）
- 加密算法由谷歌定义的算法，换成更通用的TLS
- QUIC不可靠传输的扩展
- 可路由的Connection ID
- 截止时间可感知的QUIC扩展
- ……

本章总结

➢传输服务原理:
- 传输层复用和分用
- 可靠数据传输
- 流量控制
- 拥塞控制

➢Internet中的传输服务:
- UDP
- TCP